
GraphInstruct: A Progressive Benchmark for Diagnosing Capability Gaps in LLM Graph Generation

Zihe Wei

School of Computer Science & Technology
Tongji University
weizihe@tongji.edu.cn

Sheng Xiang*

School of Computer Science & Technology
Tongji University
xiangsheng218@gmail.com

Ying Zhang

School of Computer Science & Information Technology
School of Statistics & Mathematics
Zhejiang Gongshang University
ying.zhang@zjgsu.edu.cn

Changjun Jiang

School of Computer Science & Technology
Tongji University
cjjiang@tongji.edu.cn

Abstract

Graph-structured data underpins applications from citation analysis and social-network modeling to molecular design and knowledge-graph construction, and Large Language Models (LLMs) are increasingly used as prompt-driven graph synthesizers. Classical graph-generation reviews catalog deep generative models and their evaluation primitives, but predate the LLM era and provide no foundation for evaluating instruction-following graph synthesis. Recent LLM-era benchmarks evaluate models along graph-type or task-domain axes; such organizations, however, average over structural complexity and cannot localize *where* in the complexity spectrum an LLM breaks down. To close this diagnostic gap, we introduce GraphInstruct, a progressive-complexity benchmark that stratifies LLM graph generation into six complexity levels and five evaluation dimensions, paired with 800 hand-authored instructions, 1,582 algorithmically synthesized reference solutions, and a 12-LLM capability evaluation across 45 (model, strategy) configurations. We find that discriminative power peaks at multi-constraint composition rather than reasoning depth, that no single prompting strategy dominates across levels or model families, and that domain-semantic constraints remain iteration-invariant under all tested methods—pointing to retrieval rather than additional compute as the next research frontier. Atop the benchmark, a verification-guided iterative framework with constraint-aware adaptive prompting consistently surpasses the prompt-engineering ceiling on tested target models, demonstrating that the benchmark’s fine-grained signals drive method development. Data, code, and reproducibility artifacts are included in the supplementary materials and will be released publicly upon acceptance.

*Corresponding author.

1 Introduction

Challenges. Graph-structured data underlies an ever-expanding set of scientific and industrial applications, from citation (Sen et al., 2008; Hu et al., 2020) and social-network mining (Leskovec et al., 2007) to drug-design (Irwin et al., 2012; Ramakrishnan et al., 2014) and knowledge-graph construction, and Large Language Models (Vaswani et al., 2017; Brown et al., 2020; OpenAI, 2023; Touvron et al., 2023) are increasingly used as on-demand graph synthesizers (Wang et al., 2024; Yao et al., 2024; Fatemi et al., 2024; Wang et al., 2023a). The difficulty is not a lack of benchmarks but a *mismatch between benchmark organization and diagnostic need*: existing benchmarks stratify along axes that all average over the phenomena of interest. *Graph type* (Yao et al., 2024; Fatemi et al., 2024) conflates type recall with joint constraint satisfaction; *task domain* (Demirci et al., 2025; Peng et al., 2026) conflates structural capability with domain-knowledge retrieval; *classical graph problems* (Tang et al., 2025; Wang et al., 2023a; Chen et al., 2024) measure reasoning *about* pre-specified graphs rather than generation of new ones. The aggregate quality number each axis yields is silent on which sub-capability a proposed method actually improves: a model scoring 0.83 overall might have 0.95 single-constraint capability but 0.58 multi-constraint composition, or +4% from chain-of-thought (Wei et al., 2022; Kojima et al., 2022) on one family but −4% on another.

Motivation. Evaluation must be stratified along the structural-complexity axes that govern failure—a methodological gap, not a reporting issue. We organize evaluation along a *progressive complexity axis* from pure format emission (L0) to multi-step graph editing (L5), with four intermediate levels each introducing a new constraint type: single explicit constraint (L1), multi-constraint composition (L2), numerical-attribute control (L3), and domain semantics (L4). Each level is *progressive, not hierarchical*: a good L5 output need not satisfy an L1 tree constraint, enabling independent per-level measurement of distinct capability axes. Within each level we score outputs on five complementary dimensions—structural fidelity (D1), textual similarity (D2), embedding-based distributional proximity (D3), instruction-match adherence (D4), and token efficiency (D5)—disentangling structural correctness from surface similarity that aggregate scores conflate—two models failing on D1 vs. D4 can converge to the same aggregate, hiding which sub-capability bounds each.

Benchmark and findings. GraphInstruct comprises 800 hand-authored instructions from 40 templates, 1,582 algorithmically synthesized reference solutions (two per feasible instruction), all round-trip verified through a 418-unit-test parser. A 12-LLM capability evaluation spans 45 (model, strategy) configurations $\times$ 800 instructions $\times$ 5 generations $\approx$ 180K outputs (11 models on all four prompting strategies; Sonnet-4 on zero-shot only). The per-level $\times$ per-strategy $\times$ per-model matrix surfaces six findings: **(F1)** discriminative power peaks at *constraint composition*, not reasoning depth—T1–T3 gap reaches 0.219 at L2 ($3 \times$ L3, $2 \times$ L5); **(F2)** prompt sensitivity inversely scales with base capability (σ_{strat} drops from 0.074 weakest-T3 to 0.019 stablest-T2; $\beta = -0.27$, $R^2 = 0.62$, $p < 10^{-3}$); **(F3)** no universal prompting strategy exists—FS adds +0.069 at L4 but subtracts −0.034 at L2, FC boosts L5 by +0.045 but harms L3 by −0.048; **(F4)** CoT has model-family-specific polarity (Qwen3.5 +0.04 to +0.05, GPT-family −0.02 to −0.04, sign robust across $\sim 10 \times$ parameter gaps within each family); **(F5)** capability decouples from scale on locally-reducible tasks (Qwen3.5-35B vs. 397B on L5 inside $N=50$ CI; GPT-3.5 outperforms GPT-4.1 on L3 by $\Delta=0.049$); **(F6)** a cost axis defines a capability floor—only 6 of 45 configurations are Pareto-optimal (all Anthropic / OpenAI); 3 of 12 models never reach $Q \geq 0.8$.

Methods. These findings emerge *without any method intervention*—the benchmark is purely a diagnostic instrument. Their fine per-constraint granularity also drives method development: a three-component pipeline—Verification-Guided Iterative Generation (VGIG, inspired by text-domain self-refine (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024) but grounded in programmatic verification rather than LLM self-critique (Huang et al., 2024)), Constraint-Aware Adaptive Prompting (CAAP), and curated L4 domain priors—surpasses the empirical Oracle (per-level best of the four prompting strategies, an upper bound for any prompt-only method) by +0.035–+0.050 across three target models. Three ablations establish that (a) iterative refinement saturates at $T \approx 5$ rounds on verifiable graph constraints, much shorter than text-domain $T=10$ –20 defaults; (b) feedback *richness* dominates iteration *count*, with binary pass/fail capturing 75% of the gain; (c) L4 domain-semantic constraints are iteration-invariant across all 24 tested (T, feedback) configurations, pointing to retrieval- and grounding-based methods rather than more iteration.

Contributions. Our contributions can be summarized as follows:

- Our benchmark is the first work that occupies all six diagnostic benchmark design axes in terms of LLM-era graph generation tasks.
- Our benchmark contains progressively complex benchmarks, covering 12 instruction-tuned LLMs, 45 configurations, and approximately 180k capability evaluation instances.
- We derive a set of capability-level findings from the benchmark, and propose two improvement methods validated by comprehensive ablation studies.

Among the eleven prior graph-LLM benchmarks tabulated in §2 (Table 1), no single one occupies more than three of the six diagnostic-benchmark design axes; GraphInstruct occupies all six axes. All data, ~262K responses, code, and reproducibility artifacts are available at https://anonymous.4open.science/r/GraphInstruct_formal-3272.

2 Related Work

We position GraphInstruct against eleven prior graph-LLM benchmarks along six axes a diagnostic graph-generation benchmark must satisfy: *graph generation* (vs. computation/encoding/training), *complexity-aware stratification*, *multi-dimensional evaluation* (≥ 4 distinct dimensions), *LLM coverage* (≥ 10 frontier models), *algorithmically synthesized references*, and *built-in improvement methods*. **No prior benchmark satisfies all six axes** (Tab. 1); GraphInstruct is the first to occupy all six (static graph generation; 6-level L0–L5 stratification; 5 D1–D5 dimensions; 12-LLM survey; 1,582 algorithmically synthesized references; VGIG/CAAP/Combined methods). The concurrent GDGB (Peng et al., 2026) targets dynamic text-attributed graph generation, complementary to our static, constraint-driven setting; Demirci et al. (2025) cover a complementary domain axis (5 domains $\times$ 15 LLMs), and we quantify two of their qualitative observations (§5.1 prompt-sensitivity scaling, §5.2 CoT polarity). App. B provides prior-by-prior commentary, the axis-by-axis comparison with Demirci et al. (2025), and the classical-generative-model / prompting / structured-decoding / broader-LLM-evaluation discussions.

Table 1: Multi-axis positioning of GraphInstruct against eleven prior graph-LLM benchmarks. ✓ = supported; ✗ = absent; ◯ = partial / limited coverage. “given” indicates a hand-curated ground-truth (no algorithmic synthesis); “training” indicates an instruction-tuning artifact rather than an inference-time method.

Benchmark (Venue)	Graph generation	Complexity stratification	Multi-dim eval (≥ 4)	LLM survey (≥ 10)	Algo. ref. solutions	Improvement methods
Bonifati et al. (Bonifati et al., 2020) (CSUR’20, survey)	✓ classical	✗	◯ classical	✗ pre-LLM	n/a	✗
Xiang et al. (Xiang et al., 2022) (VLDBJ’21)	✓ classical	✗ by gen. type	✓ 17 metrics	✗ pre-LLM	12 real graphs	✓ improv.+platform
NLGRAPH (Wang et al., 2023a) (NeurIPS’23)	✗ reasoning	◯ 8 tasks	✗ accuracy	◯ 4 LLMs	given	◯ prompt methods
TALK-LIKE-A-GRAPH (Fatemi et al., 2024) (ICLR’24)	✗ encoding	◯ enc. $\times$ task	✗ accuracy	◯ 5 PaLM only	given	✗
INSTRUCTGRAPH (Wang et al., 2024) (ACL’24)	✗ training	✗	✗ task-specific	◯ 4 baselines	given	◯ training
GRAPHWIZ (Chen et al., 2024) (KDD’24)	✗ reasoning	◯ 9 problems	✗ accuracy	◯ 5 baselines	given	◯ training
LLM4GRAPHGEN (Yao et al., 2024) (arXiv’24)	✓	◯ 3 axes	◯ Valid+Novel	◯ 3 LLMs	✗	✗
GRAPHARENA (Tang et al., 2025) (ICLR’25)	✗ computation	◯ 10 problems	◯ 4 outcome cats.	✓ 10 LLMs	ground truth	◯ 4 strategies
Demirci et al. (2025) (ACL’25)	✓	◯ 5 domains	✗ 3 errors	✓ 15 LLMs	✗	✗ observational
GDGB (Peng et al., 2026) (ICLR’26)	✓ dynamic	◯ tasks	◯ 3 cat. multi-metric	◯ 4 LLMs	◯	✗
GraphInstruct (ours)	✓ static	✓ 6 L0–L5	✓ 5 D1–D5	✓ 12 LLMs	✓ 1,582 algo.	✓ VGIG/CAAP

3 Models, Datasets, and Metrics

3.1 Models and Prompting Strategies

We evaluate **12 LLMs** across three capability tiers under three criteria: ≥ 3 models per tier, 5 commercial + 2 open-source providers, and same-family scale series (§5.2). **T1** ($Q > 0.87$): Sonnet-4.6 (Anthropic, 2024), Qwen3.5-397B-A17B, Qwen3.5-122B-A10B (Yang et al., 2024b,a). **T2** ($0.82 \leq Q \leq 0.87$): Qwen3.5-35B-A3B, GPT-4.1, GPT-4o (Brown et al., 2020; OpenAI, 2023), DeepSeek-V3 (DeepSeek-AI, 2024), Llama-3.3-70B (Touvron et al., 2023; Grattafiori et al., 2024), Sonnet-4. **T3** ($Q < 0.80$): GPT-3.5-turbo, GPT-4o-mini, Llama-3.1-8B. Each model is evaluated under four prompting strategies: **zero-shot (ZS)**, **few-shot (FS)** (Brown et al., 2020) (3 same-level demonstrations), **zero-CoT (ZC)** (Kojima et al., 2022) (“Let’s think step by step”), **few-CoT (FC)** (Wei et al.,

2022) (FS+CoT demonstrations), all with the InstructGraph code-style prefix (Wang et al., 2024). Reasoning-specialized models (o1, o3-mini, DeepSeek-R1, Claude 3.7 extended thinking) are excluded as same-family scale-comparability (§5.2) requires base-model homogeneity. For each (model, strategy, instruction) cell we draw 5 independent generations at $T=0.7$, max_tokens 16,384, fixed seed; Sonnet-4 receives zero-shot only as an efficiency-baseline reference ($Q=0.834$, T2). The survey comprises 45 (model, strategy) configurations yielding $11 \times 4 \times 800 \times 5 + 1 \times 1 \times 800 \times 5 = 180,000$ outputs; four-strategy analyses use the 11 fully-evaluated models, configuration-level analyses include Sonnet-4 zero-shot as a single point.

3.2 Reference Solutions: Dual-Pool Construction

GraphInstruct rests on two distinct pools playing complementary metric roles. *Shared distributional pools* (4,163 graphs) serve as population baseline for MMD.D/C/O/S (D1) and embedding MMD (D3): an L3 synthetic pool of 3,115 graphs covers 15 attribute subgroups (density, clustering, path length, max degree, diameter, and pairwise combinations) crossed with three size buckets (small ≤ 20 , medium 21–50, large > 50); an L4 real-world pool of 1,048 graphs spans 9 domains—DBLP co-authorship (Leskovec et al., 2007), Cora/Citeseer (Sen et al., 2008), Reddit (Hamilton et al., 2017), Karate Club, Facebook ego, ZINC (Irwin et al., 2012), QM9 (Ramakrishnan et al., 2014), ecological, infrastructure, KG—all cleaned, deduplicated, size-normalized. *Quality references* (1,582 graphs) provide per-instruction instance-level grounds for D2/D3/D4, and double as FS/FC exemplars; for each of 791 feasible L3+ instructions we synthesize two constraint-satisfying graphs through per-constraint generators (random-labeled-tree, bipartite $G(n_1, n_2, p)$, k -core peeling, cyclic edge augmentation, etc.). All 1,582 references pass round-trip parse→check→serialize→re-parse via the 418-unit-test parser. **Nine L2 instructions** carry deliberate infeasible labels (e.g. “5-regular bipartite, partitions $\{3, 7\}$ ”); we retain them as a *confabulation detector*, scored on D4 alongside others. Per-pool licenses and worked examples are in App. C.

3.3 Evaluation Metrics

We score each output on five complementary dimensions; the active subset varies by level. **D1 Structural quality** uses a *type-aware activation split*: on constraint-driven levels (L0, L1, L2, L5) we report Valid Rate, Graph Edit Distance, and Uniqueness; on distribution-driven levels (L3, L4) we additionally report four MMD metrics (Gretton et al., 2012) following You et al. (2018)—degree (MMD.D), clustering (MMD.C), orbit-count (MMD.O), spectral (MMD.S). This asymmetry avoids metric pollution: GED to a closest-reference is uninformative when the reference is itself one sample from a larger distribution; a valid-but-atypical graph on a distribution-driven level should score low MMD even with high Valid-Rate. **D2 Textual quality** (level-aggregate weight non-zero only at L4 where references carry domain text labels): G-BERTScore (Zhang et al., 2020; Devlin et al., 2019), G-BLEU (Papineni et al., 2002), G-ROUGE (Lin, 2004), Text-F1 on the serialized graph string—adapting standard text-similarity primitives to graph-as-string output, in the spirit of LLM-based graph-judge proposals (Huang et al., 2025a). **D3 Embedding quality** (active L3+): Grassmann coherence (reference-free), node-classification gap (reference-based, lightweight GCN (Kipf and Welling, 2017) on real vs. generated), embedding MMD (Gretton et al., 2012; You et al., 2018). **D4 Instruction match** (active at every level, supplies the feedback signal for VGIG; §4.3): *Explicit-Constraint Satisfaction*, *Implicit Inference* (downstream-derivable constraints—e.g. tree implies connected/acyclic/ $n-1$ edges), *No-Contradiction*.

D5 Token efficiency. Targeting valid graphs (not raw emissions) and weighting token cost above API-call overhead (streaming pricing is per-token-dominated), we combine TPV and API-call count and define total Quality and Pareto-adjusted final score (Eqs. 1–3):

$$D_5 = 0.7 e^{-\text{TPV}/1000} + 0.3 e^{-(\text{API}-1)/2}, \quad (1)$$

$$Q \equiv S_{\text{total}} = \sum_{\ell=0}^5 w_{\ell} S_{\ell}, \quad w=(0.05, 0.10, 0.15, 0.20, 0.25, 0.25), \quad (2)$$

$$S_{\text{final}} = S_{\text{total}}(1 + \lambda \text{ParetoBonus}), \quad \lambda=0.15. \quad (3)$$

The exponent scales (1000 tokens; 2 extra API calls) reflect deployment thresholds, in line with HELM’s efficiency-as-evaluation-dimension framing (Liang et al., 2023); a zero-token output yields $D_5=1$. The level-weight schedule encodes a prerequisite ordering—lower levels are prerequisite,

higher levels differentiate—with arithmetic up-weighting ($w_5=5w_0$, not $10w_0$) and a plateau $w_4=w_5$. ParetoBonus $\in \{0, 1\}$ takes 1 iff the (model, strategy) is non-dominated in $\langle \text{TPV}, S_{\text{total}} \rangle$; $\lambda=0.15$ is the smallest value at which every Pareto-optimal configuration outranks its nearest-quality off-frontier peer on S_{final} . Rank stability holds under $\pm 50\%$ D5-weight perturbations, the exponential scales ($s_T=1000, s_A=2$) themselves are robust to $2\times$ perturbation (Spearman $\rho \geq 0.97$ across a 3×3 scale grid; App. D, Tab. 6), and $\lambda \in \{0.05, \dots, 0.25\}$ shifts at most 3 top-15 positions; uniform-weight ablation confirms top-9 stability (App. D). We report **Quality** Q (D1–D4) as capability and **Combined** (D1–D5) and S_{final} separately for cost-aware views; *strategy variance* σ_{strat} (std-dev of Q across the four strategies) summarizes prompt sensitivity. Three auxiliary efficiency metrics support §5.3: *Quality per kilo-TPV* $Q/\text{kTPV}=S_{\text{total}}/(\text{TPV}/1000)$ (scale-free efficiency); *Cost@Q=0.8* (minimum per-model TPV reaching $S_{\text{total}} \geq 0.8$, undefined if never); *Frontier Distance* $\text{FD}(c) = \max\{0, q^*(\log \tau_c) - s_c\}$ where q^* is the piecewise-linear frontier in $(\log \text{TPV}, S_{\text{total}})$ space ($\text{FD}=0$ iff Pareto-optimal; isolates headroom from cost).

4 The GraphInstruct Benchmark

GraphInstruct comprises three interlocking components (architecture diagram in App. A, Fig. 1): a **Progressive Instruction Layer** organizing 800 instructions into six complexity levels L0–L5; a **Five-Dimensional Evaluation Metrics** module (§3.3); and a **Cost-Effectiveness Pareto Analysis** layer combining quality and efficiency into deployment-oriented rankings. LLM-as-Judge evaluation and Multi-Agent Collaborative generation are explicit non-goals of this release (§6). This section develops design principles (§4.1), the six complexity levels (Tab. 2), instruction-construction pipeline (§4.2), and benchmark-driven generation methods (§4.3).

4.1 Design Principles

Three principles underpin the architecture. **(P1) Progressive constraint-type stratification:** levels stratify by structural constraint *type*, not by a monotonic difficulty scalar; $L0 \rightarrow L5$ introduces at each level a structurally new constraint type (format $\rightarrow$ single-explicit $\rightarrow$ multi-constraint $\rightarrow$ numerical $\rightarrow$ semantic $\rightarrow$ editing). The progression is *progressive, not hierarchical*: a good L5 output need not satisfy an L1 tree constraint, and the empirical discrimination peak is L2 (§5.1), not the highest level. **(P2) Multi-dimensional evaluation:** outputs are scored on five complementary dimensions (§3.3). Three choices follow: D3 activates at L3+; D2 carries non-zero level-aggregate weight only at L4 (other L3+ levels lack labelled-text references); D4 is mandatory at every level; D5 is reported separately so capability and deployment views do not contaminate each other. **(P3) Reference-based and reference-free evaluation:** at L3+, each feasible instruction is paired with 2 references for reference-based metrics; at L0–L2 constraint satisfaction is checkable against the instruction alone; a reference-free structural metric (Grassmann coherence) supplements D3 at all levels. The six levels are summarized with example instructions in Tab. 2.

Table 2: Six levels of GraphInstruct with illustrative instructions.

Level	# Instr.	Core challenge	Sample constraint types and example	Active dims
L0	100	Format generation	node/edge count, syntax (“10 nodes, 12 edges”)	D1, D4, D5
L1	200	Single explicit constraint	graph_type (“a tree with 10 nodes” $\rightarrow$ acyclic+connected+ $n-1$ edges)	D1, D4, D5
L2	200	Multi-constraint composition	≥ 4 joint (“directed, connected, 15 nodes, 22 edges, min-deg ≥ 2 ”)	D1, D4, D5
L3	150	Numerical attribute control	density, clustering, path length (“density ≈ 0.21 , clustering ≈ 0.35 ”); 3,115 pool	D1–D5
L4	100	Domain semantics	social/citation/molecular (“small-world social network, $\gamma \approx 2.3$ ”); 1,048 pool	D1–D5
L5	50	Multi-step graph editing	base graph + ordered edits (“ $G_0=5$ -path; add 3 to form cycle, remove longest edge”); partial credit	D1–D5

4.2 Instruction Construction and Quality Assurance

A three-stage pipeline produces the instructions. **Stage 1 (Template authoring)**: 40 hand-designed templates cover L0–L5 with parameter slots; two authors reviewed each for linguistic clarity and constraint well-formedness, rewriting any flagged as ambiguous. **Stage 2 (Parameter sampling)**: stratified sampling yields 800 instructions with balanced coverage of graph types, sizes (small ≤ 20 , medium 21–50, large > 50), and constraint-count distributions; every per-level $\times$ per-size cell contains ≥ 15 instances. **Stage 3 (Reference synthesis)**: for each feasible instruction, 2 reference graphs are synthesized by constraint-satisfying algorithms (NetworkX `random_labeled_tree` for L1-tree, `bipartite_random_graph` for L1-bipartite, k-core extraction for L2, calibrated attribute-sampling for L3). All 1,582 references pass a round-trip parse–serialize test. 9 of 800 L2 instructions are provably infeasible (regular-degree constraints incompatible with node/edge counts); we retain them as an explicit *infeasibility stress test*. 418 unit tests cover the parser, validators, all D1–D5 metrics, scoring, data loader, and evaluation pipeline; six review rounds over four weeks by different authors gated release on three consecutive clean passes (dataset overview in App. A, Fig. 2).

4.3 Benchmark-Driven Generation Methods

The benchmark’s main value is diagnostic, but its fine-grained failure signals invite a direct follow-up—*do these signals drive targeted improvement?* Five surveyed failure modes motivate three method components: F1 multi-constraint collapse + F2 numerical drift + F5 editing imprecision $\rightarrow$ **VGIG** (verification-guided iteration); F3 prompting self-bias $\rightarrow$ **CAAP** (constraint-aware adaptive prompting); F4 domain-semantic gap $\rightarrow$ **Domain Priors** (L4-only). **VGIG** iterates generation with *programmatic* feedback from D4 checkers (no LLM judge, eliminating evaluator-generator self-consistency pitfalls (Huang et al., 2024)); each round formats the violation list V_t at granularity $g \in \{\text{none, coarse, fine}\}$ and re-prompts, terminating on $V_t = \emptyset$ or $t = T$ (per-level templates Tab. 11, App. E). **CAAP** selects prompting strategy per instruction via a 168-cell (level $\times$ dominant-constraint-type $\times$ model-tier) decision table learned from the capability evaluation, with constraint-type overrides. **Domain Priors** inject eight hand-curated structural priors (Barabási and Pósfai, 2016; Newman, 2018) (degree exponent, clustering, motifs) as auxiliary L4 constraints. The **Combined** pipeline composes CAAP $\rightarrow$ VGIG $\rightarrow$ Domain Priors with modular components; §5.4 reports per-component contributions, with three-model consistency and ablation-robustness filters; full algorithms, 168-cell table, L4 prior details, and per-level VGIG feedback templates (Tab. 11) are in App. E.

5 Evaluation

We probe GraphInstruct from two ends: as a *diagnostic instrument* (§5.1–5.3, the 45-configuration capability evaluation) and as a *development platform* (§5.4, methods atop benchmark signals). Each subsection consolidates two to four research questions whose setups, per-RQ tables, mechanism analyses, case studies, and figures are migrated to App. F (per-RQ navigation: RQ1/2 F.1, RQ3/4/5 F.2, RQ6 F.3, RQ7–10 F.4). The capability profiles and full leaderboards are App. F.5.

Scale and reproducibility. The capability evaluation produces 45 (model, strategy) configurations $\times$ 800 instructions $\times$ 5 samples = 180,000 outputs (Sonnet-4 is zero-shot-only, §3.1). Method experiments span three target models (GPT-4o-mini, DeepSeek-V3, Qwen3.5-35B; plus a Qwen3.5-35B-nothink variant) across 6 conditions (VGIG, CAAP, Combined, Oracle, retry, SC), the E5 rounds ablation ($T \in \{1, 2, 3, 5, 7, 10, 15, 20\}$), and the E6 feedback-granularity ablation, totalling $\sim 262\text{K}$ outputs. All API calls are logged; generations and per-instance quality scores in `results/{model}-{strategy}.jsonl, quality.json`; hyperparameters in `configs/`; `reproduce.sh` regenerates every figure and table (App. H). Findings are flagged as robust only if effect size exceeds the ± 0.005 noise band established by the E5 rounds ablation (§5.4); sub-noise observations (e.g. $T=15$ peak, even-round patterns) are reported in App. F.4 but not elevated.

5.1 Capability stratification: where complexity bites and prompts swing

Where in the complexity spectrum does discrimination peak? For each of the six levels we compute mean Quality per capability tier (T1: 12 cells, T2: 21 incl. Sonnet-4 zero-shot, T3: 12 cells), averaged over the 45 (model, strategy) configurations within each tier, and define the tier

gap as $Q_{T1} - Q_{T3}$. Discrimination is sharply localized: **the L2 (multi-constraint composition) gap is 0.219**—3× the next-widest reasoning-heavy gap (L3, 0.073) and 2× the multi-step-editing gap (L5, 0.106) (per-level table App. F.1, Tab. 12 and Fig. 3). The mechanism is *compositional*, not reasoning-depth: each added constraint multiplies the probability of violation, and T3 models lack the working-memory capacity to maintain joint satisfaction. L2 failure is therefore not merely harder, but *brittler*: per-instruction D1 variance at L2 ($\sigma=0.240$) is 2.2× L3 and 2.4× L4, with weak models reaching $\sigma_{L2}>0.43$ while T1 models hold $\sigma_{L2}<0.13$ (App. F.1, Fig. 4). T3 models are one constraint-combination from collapse at L2; T1 models are robust to the same combinations—directly motivating per-constraint-type method gains on L2 rather than level-wise averages. A worked L2-143 case study (3×5 grid; Sonnet-4.6 satisfies all 7 constraints, GPT-4o-mini emits 19 nodes violating 4 of 7) is in App. F.1 (Fig. 5). Benchmarks averaging over constraint count *systematically underestimate* structured-generation discriminative power; method research targeting structural improvement should report L2 gains as the primary signal.

How does prompt sensitivity scale with capability? Demirci et al. (2025) report iterative-feedback gains varying 48%/ < 5% across models without an explanatory variable. We test whether this heterogeneity is predictable from base capability. For each of the **11 fully-evaluated models** (Sonnet-4 excluded as zero-shot-only) we compute strategy variance σ_{strat} , the standard deviation of Q across four prompts. T3 sits at $\sigma_{\text{strat}} \in \{0.048, 0.070, 0.074\}$ (Llama-8B, GPT-4o-mini, GPT-3.5); T2-stable hits $\sigma_{\text{strat}} \in \{0.018, 0.019\}$ (Llama-70B, DeepSeek-V3); T1 occupies $\sigma_{\text{strat}} \in \{0.040, 0.043, 0.050\}$. OLS regression of σ_{strat} on mean Q yields $\beta=-0.27$, $R^2=0.62$, $p<10^{-3}$ (Fig. 6, App. F.1). Weak models occupy an under-trained region of the output manifold where small prompt perturbations—adding a CoT trigger, switching from zero-shot to few-shot—produce large quality displacements; strong models occupy a flatter local surface. Two implications: prompt-engineering budgets should scale *inversely* with capability (for frontier models, strategy choice accounts for $\leq 2\%$ of variance—verification or retrieval yields better returns), and single-strategy benchmark evaluations systematically disadvantage prompt-sensitive models.

5.2 Strategy and family effects: no universal recipe; CoT is family-polarized; scale decouples

No single prompting strategy dominates across levels. Per-level strategy effects relative to zero-shot, averaged over the 11 fully-evaluated models (Sonnet-4 excluded because non-zero-shot strategies are undefined for it; full table App. F.2, Tab. 13; full heatmap Fig. 7), reveal opposite-signed effects across levels: **few-shot is net-negative at L2** (−0.034, the demonstration’s specific topology biases generation away from the target’s distinct constraints) and **net-positive at L4** (+0.069, domain examples convey structural priors the instruction alone cannot); **few-CoT is net-positive at L5** (+0.045, explicit step planning for edits) and **net-negative at L3** (−0.048, extraneous reasoning amplifies numerical drift). Zero-CoT is the only strategy with non-negative effect at every level, though its gains are modest where other strategies excel. Aggregate benchmarks systematically mask these opposite-signed effects: prompting-method papers reporting only aggregate gains may be silently trading L2 for L4 or L3 for L5. Level-stratified reporting should become standard on structured tasks, and strategy-per-level adaptive prompting is empirically motivated—directly informing our CAAP component (§4.3).

CoT polarity is family-bound, not capability-bound. Across the seven family-aligned models, few-CoT is uniformly beneficial for Qwen3.5 across scales (35B/122B/397B: +0.052/+0.040/+0.050) and *uniformly detrimental* for the GPT family across scales (3.5/4o-mini/4o/4.1: −0.042/−0.038/−0.005/−0.002) (App. F.2, Tab. 14 and Fig. 8). Sign robustness within each family across $\sim 10\times$ parameter gaps—GPT-4.1 agrees with GPT-3.5 despite roughly two orders of magnitude of capability; Qwen3.5-35B agrees with Qwen3.5-397B despite a $\sim 10\times$ parameter gap—indicates CoT effectiveness is governed by **pretraining distribution** rather than capability tier. We conjecture Qwen3.5’s heavy code/mathematical/reasoning-chain mix creates a CoT prior that transfers positively to graph generation, while GPT-family’s broader-but-less-code-heavy diet creates a weaker or negatively-transferring prior; direct verification would require pretraining-mixture ablations beyond our scope. The text-domain CoT folk wisdom (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023b) fails to generalize from GPT to Qwen3.5; prompting-method claims derived from one family require independent validation on the other.

Parameter scaling decouples from per-level capability. Same-family scaling (Qwen3.5 at 35B/122B/397B, GPT at 3.5/4o-mini/4o/4.1; full per-level decoupling table App. F.2, Tab. 15;

per-level curves Fig. 9) shows aggregate scaling-law narratives mask per-level heterogeneity: scaling Qwen3.5 from 35B to 397B adds +0.032 at L3 and +0.024 at L4, but *subtracts* 0.005 at L5—tasks reducing to local structural operations saturate at smaller scales than global-constraint reasoning. The GPT counterpart is sharper: **GPT-3.5 outperforms GPT-4.1** on L3 numerical attributes (0.808 vs. avg 0.759, $\Delta=-0.049$), a reversal aggregate quality conceals entirely. The reversal interacts with RQ4—larger GPT-family models’ CoT inclinations produce longer, more error-prone chains on tasks smaller models attempt directly. This extends the emergent-behavior critique of Schaeffer et al. (2023) to a specific structured-generation sub-capability and supports parameter-efficient “smaller model for subtask A, larger for subtask B” deployment strategies aggregate benchmarks obscure.

5.3 Cost-aware deployment: a 6-point provider-concentrated frontier

Quality alone fails to capture whether a +3% gain at $5\times$ cost is deployment-rational. Applying the benchmark’s efficiency instruments— S_{final} (Eq. 3), $Q/k\text{TPV}$, and $\text{Cost}@Q=0.8$ —to the 45 baselines, three results emerge (full Pareto-frontier, S_{final} -rank, $\text{Cost}@Q=0.8$, and Frontier-Distance tables in App. F.3, Tables 16–18; Frontier-Distance distribution Fig. 11).

Result (a): A 6-point provider-concentrated Pareto frontier. Only **6 of 45** configurations are non-dominated (Fig. 10; Tab. 16), all Anthropic or OpenAI; no Qwen3.5/DeepSeek/Llama configuration is Pareto-optimal despite near-frontier candidates (Qwen3.5-397B zero-CoT reaches $Q=0.862$ at $\text{TPV}\approx 2300$, dominated by Sonnet-4.6 zero-CoT at $\text{TPV}=969$, $Q=0.878$). The frontier traverses a strategy progression: zero-shot at the low-cost end (GPT-4o $\rightarrow$ Sonnet-4 $\rightarrow$ Sonnet-4.6), zero-CoT/few-shot in the middle, few-CoT on Sonnet-4.6 at the high-cost end. Two additional providers sit a hair’s-breadth from the frontier ($\text{FD}<0.02$: Llama-3.3-70B zero-shot, GPT-4.1 zero-shot); the five largest FD values all combine T3 capability with few-CoT or few-shot, confirming weak-capability models *compound* their quality deficit under expensive prompting.

Result (b): S_{final} reshuffles the leaderboard. The Pareto bonus moves 6 configurations up by 5–18 positions and pushes high-cost Qwen3.5 runs down by 2–4: GPT-4o zero-shot climbs $S_{\text{total}}\#24\rightarrow S_{\text{final}}\#6$ (+18), Qwen3.5-397B few-CoT falls $\#3\rightarrow\#7$. Among the 11 models for which per-model strategy comparison is defined, the S_{total} -best strategy flips for **only GPT-4o** (few-shot $\rightarrow$ zero-shot); the other 10 retain their S_{total} -optimal choice. On $Q/k\text{TPV}$, the top five are all zero-shot (GPT-4o 1.43, Llama-3.3-70B 1.38, GPT-4.1 1.36, Sonnet-4 1.34, Sonnet-4.6 1.31), and **no few-CoT enters the top-15** despite few-CoT being the S_{total} -optimal strategy for 5 of 11 fully-evaluated models.

Result (c): A hard capability floor at $\text{Cost}@Q=0.8$ (Pareto-frontier visualization in App. F.3, Fig. 10). Eight of the 11 fully-evaluated models cross $Q\geq 0.8$ under at least one strategy (nine of 12 including Sonnet-4 zero-shot); **zero-shot is the cheapest threshold-crossing strategy in every row**. The three cheapest paths come from structurally different T2 providers (GPT-4o 578, Llama-3.3-70B 590, GPT-4.1 599) plus Sonnet-4 zero-shot at 625, defining a $\text{TPV}\approx 600$ deployment floor. Three T3 models—**GPT-3.5, GPT-4o-mini, Llama-3.1-8B**—**never** cross under any of the four prompting strategies, marking an empirical capability floor that prompting alone cannot cross. Sonnet-4.6 zero-shot already dominates every T2 model’s most expensive strategy on both axes; the $3.3\times$ TPV increase from Sonnet-4.6 zero-shot to few-CoT buys only +0.043 Quality, foreshadowing the ~ 5 -round refinement saturation in §5.4. We recommend reporting S_{final} as a *complementary* deployment-oriented leaderboard alongside (not replacing) S_{total} : the two answer different questions; the 3-model T3 shortfall defines the concrete capability target for the verification-guided methods that follow, where Combined lifts all three T2/T3 targets above $Q=0.85$ via feedback richness (§5.4).

5.4 Methods atop benchmark signals: verification beats prompt engineering

Can the benchmark’s fine-grained D4 signal drive method improvement *beyond* prompt-only ceilings? Let **Oracle** denote per-level best-of-four prompting (an empirical upper bound on prompt-only methods, requiring oracle knowledge of the optimal per-level strategy). On three target models (GPT-4o-mini T3, DeepSeek-V3 T2, Qwen3.5-35B T2), **Combined exceeds Oracle by +0.035 to +0.050**— $7\times$ the ± 0.005 noise band (App. F.4, Tab. 19 and Fig. 12): GPT-4o-mini 0.7523 $\rightarrow$ 0.8549 over Oracle 0.8052 ($\Delta=+0.050$); DeepSeek-V3 0.8215 $\rightarrow$ 0.8941 vs. Oracle 0.8577 (+0.036); Qwen3.5-35B 0.8092 $\rightarrow$ 0.9071 vs. Oracle 0.8720 (+0.035). Per-component decomposition shows VGIG-only contributes the majority of the gain. Prompt engineering has a measurable empirical ceiling; external programmatic verification—not prompt phrasing—is the binding mechanism for

reliable structured graph generation. The gap is largest for the weakest target (mirroring §5.1’s inverse-scaling): the low-capability regime leaves the most room for method intervention.

Sampling without verification fails. Pure retry ($T=3$, no feedback) and self-consistency ($N=3$, best-of- N) stagnate within ± 0.01 of zero-shot on all three target models—a strong null result with six controls all in-band (App. F.4, Tab. 20: GPT-4o-mini retry -0.003 /SC -0.001 ; DeepSeek-V3 $+0.005/-0.001$; Qwen3.5-35B $-0.009/-0.005$). This contrasts sharply with text-domain self-consistency (Wang et al., 2023b) (e.g. 5–10% on GSM8K (Cobbe et al., 2021)): text tasks have many plausible-looking outputs of which only some are correct (majority-vote recovers); structured-graph tasks have many wrong outputs with violating structures (majority-voting among wrong outputs cannot help). Verifiable structured tasks have a fundamentally different compute-to-quality relation; deployment budgets should be allocated to verification infrastructure, not parallel sampling.

Feedback richness, not iteration count, is the binding lever. Holding $T=3$ on GPT-4o-mini, the verify-only signal (binary pass/fail, no per-constraint detail) captures **75% of total gain** ($+0.046$ of $+0.061$); coarse and fine each contribute a further $+0.008$ monotonically (App. F.4, Tab. 21; the $\text{retry} < \text{none} < \text{coarse} < \text{fine}$ ordering holds at every level). Iterative refinement saturates at $T \sim 5$: $T \in \{5, 7, 10, 15, 20\}$ all fall within the ± 0.005 noise band (Q at $T=1, 2, 3, 5, 7, 10, 15, 20$: 0.800, 0.798, 0.810, 0.821, 0.816, 0.818, 0.827, 0.816; full curve App. F.4, Fig. 14)—markedly shorter than text-domain self-refine budgets of 10–20 (Madaan et al., 2023; Shinn et al., 2023); the standard text-domain hyperparameters should not transfer without recalibration. For benchmark design two implications follow: even a coarse verify-true/verify-false signal enables verification-driven method research, and our fine per-constraint signal unlocks the additional 25%—benchmarks lacking a programmatic verification signal are missing a method-enabling resource for the community.

L4 is iteration-invariant. Across the full T -sweep $\times$ three feedback granularities (24 configurations), L4 quality remains at 0.750–0.754—a 0.004 range (App. F.4, Fig. 15). Iteration *does* converge (within- T variance is tight) but converges to a quality ceiling prompting, iteration, and feedback granularity all fail to raise. Per-dimension decomposition reveals *why*: D1 structural validity is high and uniform across 12 models (mean 0.89, range 0.84–0.93), D3 embedding similarity is uniform (0.58–0.59), D4 instruction match is high and uniform (0.89–0.93); only **D2 text-to-reference similarity is low and extremely variable** (mean 0.050, range 0.008–0.176, $22\times$ gap, CV 1.02; App. F.4, Fig. 16). Models produce graphs that are structurally valid, constraint-satisfying, and distributionally close to real citation/social/molecular graphs but fail to reproduce the *specific surface serialization* of reference graphs from a given domain. The D4 verifier is already satisfied; no amount of verifier-guided re-generation can close this gap. This relocates L4 from a *capability* gap to a *grounding* gap: few-shot (3 in-context L4 exemplars) lifts GPT-4o-mini L4 from 0.744 $\rightarrow$ 0.798 ($+0.054$); Combined reaches 0.843 ($+0.099$)—the only intervention class breaking the iteration-only ceiling (App. F.4, Tab. 19). The L4 ceiling is also metric-bounded: D2 weight at L4 is 0.15 and observed max is 0.176, capping further D2-driven lift at $+0.022$ (App. F.4). **L4 is iteration-invariant** just as L5 is scale-invariant (§5.2)—both point to retrieval and grounding, not more iteration, as the next research frontier. The ± 0.005 noise band established here is the significance threshold throughout this paper.

Synthesis. GraphInstruct as a *diagnostic instrument* structures capability as a multi-dimensional profile along complexity level $\times$ prompting strategy $\times$ model family; as a *development platform*, the D4 signal unlocks $+0.035$ – $+0.050$ above Oracle. Per-dimension profiles are jagged and top-tier margins narrow (Sonnet-4.6 few-CoT $Q=0.902$, $+0.023$ over Qwen3.5-397B few-CoT); full per-dim tables, radars, leaderboards in App. F.5, G.

6 Discussion and Limitations

This work covers 12 instruction (non-reasoning) LLMs; same-family scale comparability (§5.1–5.2) requires base-model homogeneity, and Demirci et al. (2025)’s 5-domain reasoning suite jointly spans (complexity $\times$ domain $\times$ reasoning). **L4 frontier.** L4 stays flat across every iteration-only method (CV 1.02 on D2); few-shot’s $+0.054$ and Combined’s $+0.099$ L4 lifts on GPT-4o-mini point to grounding, not more iteration, with D2-weight arithmetic capping further D2-driven gains at $+0.022$ (App. F.4). **Limitations.** 800 instructions / 1,582 references suffice for ≥ 15 per-cell instances but not full combinatorial coverage; L5 is intentionally small ($N=50$, 95% CI ± 0.019 ; scale-invariance rests on a -0.005 Qwen3.5-35B-vs-397B gap inside both CI and noise band). Reasoning-specialized models (o1, DeepSeek-R1) are not evaluated under the homogeneity constraint, leaving open whether

reasoning training flips CoT polarity or relaxes L5 saturation. All D1–D5 metrics are deterministic, free of LLM-as-judge (Huang et al., 2024, 2025b) (App. F.4 confirms no D4-gaming).

7 Conclusion

This work presents GraphInstruct, a comprehensive benchmark for LLM-based graph generation that spans six complexity levels and five diagnostic dimensions, built on 800 structured instructions and 1,582 reference graphs. Through a large-scale evaluation of 12 LLMs, we distill six key capability-level findings that cannot be revealed by traditional single-axis benchmarks. We further propose a verification-guided optimization framework, which outperforms the strong prompt-engineering, with feedback richness identified as the critical factor for consistent improvement. Looking forward, natural and impactful future directions include: (i) expanding the L5 test set to 150 instances for higher statistical power; (ii) introducing an L6 level to support multi-graph reasoning and cross-structure inference; (iii) evaluating reasoning-specialized models (e.g., o1, DeepSeek-R1) under consistent settings; and (iv) developing structure-aware and retrieval-augmented generation methods to break the saturation barrier of iterative prompting.

Acknowledgments and Disclosure of Funding

We thank the anonymous reviewers for feedback that improved this work. Funding disclosure and competing interests: none at the time of submission. This work used large-language-model APIs from Anthropic, OpenAI, Alibaba Cloud, DeepSeek, and open-source inference infrastructure for Llama-family models; compute budgets are itemized in Appendix H.

References

- Anthropic. The Claude 3 model family: Opus, sonnet, haiku. Technical report, Anthropic, 2024.
- Albert-László Barabási and Márton Pósfai. *Network Science*. Cambridge University Press, 2016.
- Angela Bonifati, Irena Holubová, Arnau Prat-Pérez, and Sherif Sakr. Graph generators: State of the art and open challenges. *ACM Computing Surveys*, 53(2):1–30, 2020. doi: 10.1145/3379445.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Nuo Chen, Yuhan Li, Jianheng Tang, and Jia Li. GraphWiz: An instruction-following language model for graph computational problems. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024. doi: 10.1145/3637528.3672010.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Ege Demirci, Rithwik Kerur, and Ambuj Singh. Are LLMs truly graph-savvy? A comprehensive evaluation of graph generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL) Student Research Workshop*, pages 884–897, 2025.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé Iii, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021. doi: 10.1145/3458723.

- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*, 2024.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research (JMLR)*, 13:723–773, 2012.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, et al. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. Preprint title: “Training Compute-Optimal Large Language Models” (arXiv:2203.15556); the model is known as Chinchilla.
- Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can LLMs be good graph judge for knowledge graph construction? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025a.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems (TOIS)*, 43(2), 2025b. doi: 10.1145/3703155.
- John J. Irwin, Teague Sterling, Michael M. Mysinger, Erin S. Bolstad, and Ryan G. Coleman. ZINC: A free tool to discover chemistry for biology. *Journal of Chemical Information and Modeling*, 52(7):1757–1768, 2012.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and shrinking diameters. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 1(1), 2007. doi: 10.1145/1217299.1217301.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research (TMLR)*, 2023.
- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL Workshop*, 2004.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Mark Newman. *Networks*. Oxford University Press, 2nd edition, 2018.

- OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002.
- Jie Peng, Jiarui Ji, Runlin Lei, Zhewei Wei, Yongchao Liu, and Chuntao Hong. GDGB: A benchmark for generative dynamic text-attributed graph learning. In *International Conference on Learning Representations (ICLR)*, 2026.
- Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.
- Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008. doi: 10.1609/aimag.v29i3.2157.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Jianheng Tang, Qifan Zhang, Yuhan Li, Nuo Liu, Hongzhi Hua, Jiawei Jin, Yi Wang, and Xiao Huang. GraphArena: Evaluating and exploring large language models on graph computation. In *International Conference on Learning Representations (ICLR)*, 2025.
- Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.
- Jianing Wang, Junda Wu, Yupeng Hou, Yao Liu, Ming Gao, and Julian McAuley. InstructGraph: Boosting large language models via graph-centric instruction tuning and preference alignment. In *Findings of the Association for Computational Linguistics (ACL)*, pages 13492–13510, 2024.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Sheng Xiang, Dong Wen, Dawei Cheng, Ying Zhang, Lu Qin, Zhengping Qian, and Xuemin Lin. General graph generators: Experiments, analyses, and improvements. *The VLDB Journal*, 31(5): 897–925, 2022. doi: 10.1007/s00778-021-00701-5. Online-first October 2021.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024a.
- An Yang et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024b.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- Yang Yao, Xin Wang, Zeyang Zhang, Yijian Qin, Ziwei Wang, Xu Chu, Yuekui Yang, Wenwu Zhu, and Hong Mei. Exploring the potential of large language models in graph generation. *arXiv preprint arXiv:2403.14358*, 2024.
- Jiaxuan You, Rex Ying, Xiang Ren, William L. Hamilton, and Jure Leskovec. GraphRNN: Generating realistic graphs with deep auto-regressive models. In *International Conference on Machine Learning (ICML)*, 2018.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.
- Denny Zhou, Nathanael Scharli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A Framework Diagram and Dataset Overview

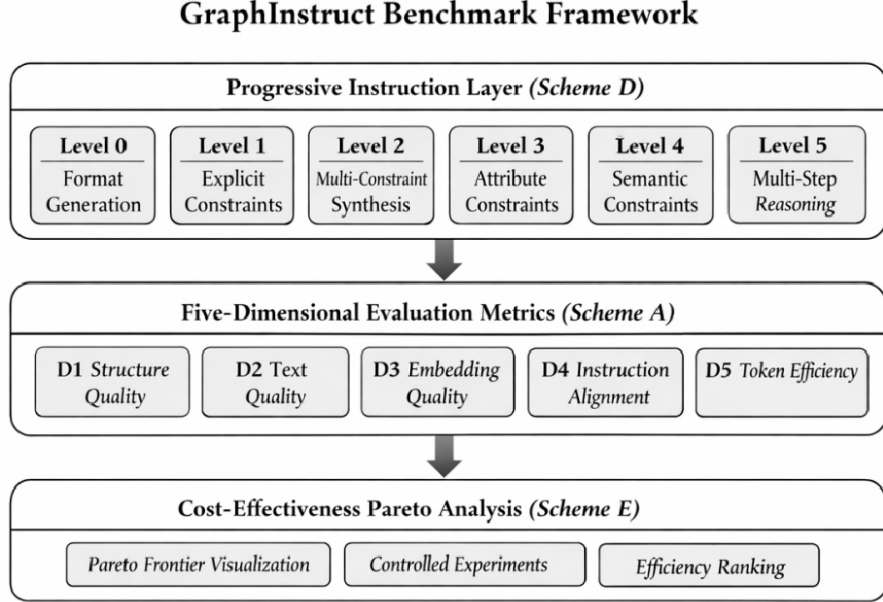


Figure 1: The GraphInstruct benchmark framework. The Progressive Instruction Layer (L0–L5) feeds into the Five-Dimensional Evaluation Metrics pipeline (D1–D5); the Pareto analysis layer integrates Quality (D1–D4) and Efficiency (D5) into deployment-oriented rankings. LLM-as-Judge and Multi-Agent Collaboration are optional future modules.

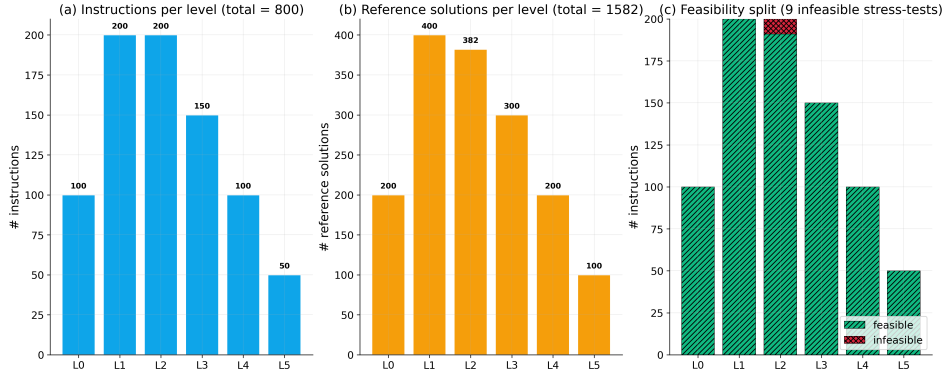


Figure 2: GraphInstruct dataset overview. Left: per-level instruction count. Center: graph-size distribution. Right: constraint-count distribution, showing progressive increase from L0–L1 to L2 (4+ constraints) and compound semantic/editing constraints at L4–L5.

B Related Work: Extended Discussion

This appendix supplements §2 with prior-by-prior commentary, the axis-by-axis comparison with Demirci et al. (2025), and the classical-generative-model / prompting / structured-decoding / broader-LLM-evaluation discussions migrated here for length. The full positioning table is Tab. 1 in §2.

Closest concurrent: GDGB and Demirci et al. GDGB (Peng et al., 2026) (ICLR’26) targets dynamic text-attributed graph generation with three task categories (continuation, retrieval, prediction) and four LLMs; it is complementary to our static, constraint-driven setting. Demirci et al. (2025) (ACL’25) cover a complementary domain axis (5 application domains $\times$ 15 LLMs); they observe

iterative-feedback gains varying 48%/ < 5% across models without identifying an explanatory variable, and qualitatively note that prompting strategies differ in effectiveness across models. Our §5.1 and §5.2 quantify two of their qualitative observations: prompt sensitivity inversely scales with capability ($\beta = -0.27$, $R^2 = 0.62$) and CoT polarity is family-specific. Axis-by-axis: their benchmark stratifies by application domain, ours by structural complexity; their generation method is prompt-only, ours adds a verification-guided pipeline; they evaluate 15 LLMs on 5 domains, we evaluate 12 LLMs on 6 levels with same-family scale series enabling RQ4/RQ5. The two benchmarks are complementary, jointly covering (complexity $\times$ domain) of LLM graph generation.

Earlier graph-generation benchmarks. LLM4GRAPHGEN (Yao et al., 2024) stratifies along three property axes (graph type, scale, hardness) but evaluates only 3 LLMs and reports two metrics (Validity, Novelty); no algorithmically synthesized references. INSTRUCTGRAPH (Wang et al., 2024) (ACL’24) introduces the code-style format we adopt and a hallucination taxonomy, but as a training/instruction-tuning artifact rather than an inference-time diagnostic. GRAPHARENA (Tang et al., 2025) (ICLR’25) evaluates 10 LLMs on 10 classical computation problems with four prompting strategies, surfacing a coverage notion of capability that aggregates over algorithmic problem types but does not stratify generation complexity. Earlier surveys (Bonifati et al., 2020; Xiang et al., 2022) predate the LLM era and frame graph generation as a classical statistical-modeling task; our work treats LLMs as on-demand graph synthesizers and asks where their failures cluster.

Reasoning-about-graphs benchmarks. NLGRAPH (Wang et al., 2023a) (NeurIPS’23) evaluates LLM reasoning on 8 classical graph problems (BFS/DFS/MST/...); TALK-LIKE-A-GRAPH (Fatemi et al., 2024) (ICLR’24) evaluates encoding $\times$ task combinations for PaLM-only; GRAPHWIZ (Chen et al., 2024) (KDD’24) trains a problem-solver model on 9 classical graph problems. All three measure reasoning *about* pre-specified graphs rather than *generation of new graphs*; our axis is distinct.

Prompting and structured decoding. Chain-of-thought prompting (Wei et al., 2022; Kojima et al., 2022) and self-consistency (Wang et al., 2023b) are text-domain primitives; we test their transfer to structured graph generation in §5.2 (CoT family polarity) and §5.4 (self-consistency null result). Self-refine (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024) and tree-of-thought (Yao et al., 2023; Zhou et al., 2023) establish a $T \sim 10\text{--}20$ refinement budget; we re-calibrate this to $T \sim 5$ on verifiable graph constraints. LLM self-critique limitations (Huang et al., 2024, 2025b) motivate our use of programmatic verification rather than LLM-as-judge.

Broader LLM evaluation. Scaling-law studies (Kaplan et al., 2020; Hoffmann et al., 2022) and emergent-behavior critiques (Schaeffer et al., 2023) contextualize our §5.2 scale-decoupling finding. The MMD evaluation primitives we adopt for D1 and D3 trace to Gretton et al. (2012); You et al. (2018); G-BERTScore/G-BLEU/G-ROUGE adapt Zhang et al. (2020); Papineni et al. (2002); Lin (2004); Devlin et al. (2019) to graph-as-string output, in the spirit of LLM-based graph-judge frameworks (Huang et al., 2025a). Grassmann coherence is implemented per its standard subspace-distance formulation.

C Worked Example per Level

We give one representative instruction per level (L0–L5), in the order they appear in `data/instructions/level_X.json`. Each entry shows the natural-language instruction, the explicit and implicit constraint specifications, the size bucket, the feasibility flag, and one of the two algorithmically synthesized reference solutions in our InstructGraph code-style format. L4 entries additionally carry a `domain` field; L5 entries additionally carry a `base_graph` field denoting the graph the LLM is asked to edit.

L0 – Format generation (L0–001).

- **instruction:** “Create a graph with 3 nodes.”
- **explicit_constraints:** [`"num_nodes=3"`]
- **implicit_constraints:** [`"directed=false"`]
- **graph_sizes:** [`"small"`]; **feasible:** true

Reference solution (1 of 2):

```
Graph[name='L0-001-ref1', nodes=3] {
  node_list = ['0', '1', '2'];
  edge_list = [('0','1'), ('0','2'), ('1','2')];
}
```

L1 – Single explicit constraint (L1-001).

- **instruction:** “Generate a tree with 5 nodes.”
- **explicit_constraints:** ["graph_type=tree", "num_nodes=5"]
- **implicit_constraints:** ["num_edges=4", "acyclic=true", "connected=true"]
- **graph_sizes:** ["small"]; **feasible:** true

Reference solution (1 of 2):

```
Graph[name='L1-001-ref1', nodes=5] {
  node_list = ['0', '1', '2', '3', '4'];
  edge_list = [('0','3'), ('0','2'), ('1','2'), ('1','4')];
}
```

L2 – Multi-constraint composition (L2-001).

- **instruction:** “Generate a connected 3-regular graph with 8 nodes.”
- **explicit_constraints:** ["degree=3", "num_nodes=8", "connected=true", "directed=false"]
- **implicit_constraints:** ["num_edges=12"] (from $|E| = \sum d_v / 2$)
- **graph_sizes:** ["small"]; **feasible:** true

Reference solution (1 of 2):

```
Graph[name='L2-001-ref1', nodes=8] {
  node_list = ['0', '1', '2', '3', '4', '5', '6', '7'];
  edge_list = [
    ('0','1'), ('0','3'), ('0','6'), ('1','5'), ('1','3'),
    ('2','7'), ('2','3'), ('2','6'), ('4','6'), ('4','5'),
    ('4','7'), ('5','7')];
}
```

L3 – Numerical attribute control (L3-001).

- **instruction:** “Generate a community-structured graph with 12 nodes divided into 2 communities using a stochastic block model. The graph should be connected, be undirected, have density at most 0.451, have modularity at least 0.25, have clustering coefficient at least 0.519, and have average path length at most 2.0213.”
- **explicit_constraints:** ["num_nodes=12", "connected=true", "directed=false", "density<=0.451", "modularity>=0.25", "clustering_coefficient>=0.519", "average_path_length<=2.0213"]
- **implicit_constraints:** [] (numerical attributes are themselves the contract)
- **graph_sizes:** ["small"]; **feasible:** true

Reference solution (1 of 2; note per-node block attributes and graph-level metadata):

```
Graph[name='L3-001-ref1', nodes=12] {
  node_list = ['0', '1', '10', '11', '2', '3', '4', '5', '6', '7', '8', '9'];
  edge_list = [
    ('0','1'), ('0','2'), ('0','3'), ('0','5'), ('1','3'),
    ('1','4'), ('1','5'), ('1','8'), ('2','3'), ('2','5'),
    ('2','7'), ('3','5'), ('3','6'), ('4','6'), ('4','8'),
    ('5','6'), ('5','7'), ('6','8'), ('6','9'), ('7','8'),
    ('7','9'), ('8','9')];
}
```

```

        ('5','6'), ('6','7'), ('6','8'), ('6','10'), ('6','11'),
        ('7','8'), ('7','9'), ('7','10'), ('7','11'), ('8','11'),
        ('9','11'), ('10','11')];

0.block=0; 1.block=0; 2.block=0; 3.block=0; 4.block=0; 5.block=0;
6.block=1; 7.block=1; 8.block=1; 9.block=1; 10.block=1; 11.block=1;

k = 2; model = 'sbm'; subgroup = 'SBM-S1';
partition = [{0, 1, 2, 3, 4, 5}, {6, 7, 8, 9, 10, 11}];
}

```

L4 – Domain semantics (L4-001).

- **instruction:** “Generate a social network with 10 users in an online community. Members form connections based on shared interests and interactions. The network should have density at most 0.7595, have minimum degree 1, be connected, and be undirected.”
- **explicit_constraints:** ["num_nodes=10", "density<=0.7595", "min_degree=1", "connected=true", "directed=false"]
- **implicit_constraints:** []; **domain:** "social"
- **graph_sizes:** ["small"]; **feasible:** true

Reference solution (1 of 2; note per-node string label attributes and graph-level domain / edge_type):

```

Graph[name='L4-001-ref1', nodes=10, domain='social'] {
  node_list = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'];
  edge_list = [
    ('0','3'), ('0','5'), ('0','6'), ('0','7'), ('0','8'),
    ('1','3'), ('1','4'), ('1','5'), ('1','6'), ('1','7'), ('1','9'),
    ('2','3'), ('2','4'), ('2','5'), ('2','7'), ('3','4'),
    ('3','5'), ('3','6'), ('3','7'), ('3','8'), ('3','9'),
    ('4','5'), ('4','6'), ('4','7'), ('5','6'), ('5','7'),
    ('5','8'), ('5','9'), ('6','7'), ('6','8'), ('7','8')];

0.label='Alice'; 1.label='Cora'; 2.label='Hugo'; 3.label='Grace';
4.label='Yuki'; 5.label='Wendy'; 6.label='Mona'; 7.label='Amber';
8.label='Hope'; 9.label='Jill';

domain = 'social'; edge_type = 'colleagues';
}

```

L5 – Multi-step graph editing (L5-024).

- **instruction:** “Given a 4-cycle (0-1-2-3) with one diagonal edge (0,2). Nodes 0 and 2 have degree 3, while nodes 1 and 3 have degree 2. Add the minimum number of edges to make the graph 3-regular (all vertices degree 3).”
- **explicit_constraints:** ["num_nodes=4", "connected=true", "degree=3", "directed=false", "task_type=make_regular"]
- **implicit_constraints:** ["num_edges=6", "directed=false"]
- **graph_sizes:** ["small"]; **feasible:** true

Base graph (the LLM is asked to transform this):

```

Graph[name='L5-024-base', nodes=4] {
  node_list = ['0', '1', '2', '3'];
  edge_list = [('0','1'), ('0','3'), ('0','2'), ('1','2'), ('2','3')];
}

```

Reference solution (1 of 2; the unique additional edge is (1,3)):

```

Graph[name='L5-024-ref1', nodes=4] {
  node_list = ['0', '1', '2', '3'];
  edge_list = [
    ('0','1'), ('0','2'), ('0','3'), ('1','2'),
    ('1','3'), ('2','3')];
}

```

L4 reference-pool provenance and licensing. The L4 reference pool comprises 1,048 graphs drawn from nine public corpora. Each subset retains its upstream license; we redistribute under terms compatible with research use. The full per-source attribution is in `DATA_LICENSE.md` accompanying the release; a summary appears in Tab. 3.

Table 3: L4 real-world reference-pool provenance summary (full per-source citations in `DATA_LICENSE.md`).

L4 sub-pool	Upstream source(s)	License / terms
<code>citation.pkl</code>	DBLP, Cora	SNAP terms; open-access
<code>social.pkl</code>	Reddit hyperlinks, Karate Club, Facebook ego	MIT / public-domain / SNAP
<code>biological.pkl</code>	ZINC, QM9, MUTAG (TUDataset)	ZINC research / CC0 / CC-BY
<code>infrastructure.pkl</code>	SNAP roadNet-PA (subset)	SNAP terms
<code>communication.pkl</code>	SNAP email-Eu-core	SNAP terms
<code>ecological.pkl</code>	Public food-web databases	Public-domain compilations
<code>general.pkl</code>	Wikidata subgraph snapshots	CC0
<code>ba-{s,m,l}.pkl</code>	Synthetic BA random graphs (ours)	CC-BY-4.0

All L4 graphs are size-normalized via BFS / random-walk subgraph sampling, deduplicated by Weisfeiler-Lehman hash, and stripped of node / edge attributes other than what D2 (token-level reference similarity) and D3 (embedding-MMD / node-classification gap) require.

D Scoring Formulas and Weight Ablation

This appendix gives the explicit per-level dimension weights used by all experiments, recaps the score formulas from §3.3, and reports a perturbation analysis of the weights and the D5 hyperparameters.

Per-level dimension weights. The level score $S_\ell = \sum_d w_{\ell,d} S_{d,\ell}$ uses the per-level dimension weights in Tab. 4. D2 (token-level reference similarity) is active only at L4, where reference graphs carry domain-specific surface text; D3 (embedding similarity) is active only at L3–L5, where reference *distributions* are well-defined. D4 (instruction match) is the largest weight at every level, reflecting the design intent that joint constraint satisfaction is the primary diagnostic signal.

Table 4: Per-level dimension weights (default scoring; rows sum to 1 within each level). “D2/D3 active” lists the discriminative metrics at each level; for the others the corresponding weight is 0.

Level	D1 (struct)	D2 (text)	D3 (embed)	D4 (instr)	D5 (eff)	D2/D3 active
L0	0.10	0.00	0.00	0.60	0.30	none
L1	0.15	0.00	0.00	0.70	0.15	none
L2	0.15	0.00	0.00	0.70	0.15	none
L3	0.15	0.00	0.15	0.50	0.20	D3
L4	0.10	0.15	0.05	0.55	0.15	D2, D3
L5	0.15	0.00	0.15	0.50	0.20	D3

Score formulas (recap). Total Quality (Eq. 2), D5 efficiency (Eq. 1), and Pareto-adjusted final score (Eq. 3) are defined in §3.3. The combined score reported throughout the paper is the level-weighted Quality, $Q \equiv S_{\text{total}} = \sum_{\ell=0}^5 w_\ell S_\ell$ with $w = (0.05, 0.10, 0.15, 0.20, 0.25, 0.25)$. Two evaluation modes are supported by `graphinstruct.scoring`:

- **Default** keeps D5 in the per-level weights as shown in Tab. 4 and feeds the cost-aware S_{final} (Eq. 3; $\lambda=0.15$ unless noted).

- **Quality-only** (the mode used for every S_{total} leaderboard in §5 and App. F–G) zeroes the D5 weight and renormalises D1–D4 to sum to 1, isolating quality from cost. The published S_{total} values are produced in this mode.

Weight perturbation analysis. We probe how sensitive the 45-cell quality-only S_{total} ranking is to alternative weight schemes by recomputing S_{total} on the same per-instruction dimension scores under several perturbations (`scripts/weight_ablation.py`). Tab. 5 reports the overlap with the default top-9, top-15, and top-20 sets, the top-15 Jaccard, and the largest position shift in the default top-15. Small perturbations (D4 weight +0.05 at every level) leave the ranking essentially unchanged. Aggressive perturbations restructure the ranking, with D4 emerging as the dominant ranking signal: a D4-only scheme retains 8 of 9 default top-9 entries, while a D1-only scheme retains only 3 of 9. Uniform weighting (D1=D2=D3=D4=0.25 at every level) sits between these extremes and shifts as many as 12 positions inside the top-15, primarily because L0–L2 lack meaningful D2/D3 signal so the uniform scheme injects noise from inactive dimensions at low levels.

Table 5: Weight perturbation analysis on the 45-cell quality-only ranking. “Default” is the per-level scheme of Tab. 4 with D5 zeroed. All schemes are quality-only (D5=0, D1–D4 renormalised to 1).

Scheme	Top-9 retained	Top-15 retained	Top-15 Jaccard	Max shift, top-15
Default	9 / 9	15 / 15	1.000	0
D4 weight +0.05 at every level	9 / 9	15 / 15	1.000	1
Uniform D1=D2=D3=D4=0.25	5 / 9	11 / 15	0.579	12
D4-only	8 / 9	12 / 15	0.667	8
D1-only	3 / 9	9 / 15	0.429	20

The pattern matches the design rationale: D4 (instruction match) carries the most diagnostic information because parser-passing graphs already get near-1 D1, and D5 captures cost rather than capability. Quality-only zeroing of D5 separates capability from cost; the cost-aware view is reported separately as S_{final} in App. F.3.

D5 exponential-scale robustness. The default D5 form $0.7 e^{-\text{TPV}/1000} + 0.3 e^{-(\text{API}-1)/2}$ uses two free hyperparameters: the TPV scale ($s_T=1000$ tokens) and the API-call scale ($s_A=2$ extra calls). To verify rank stability under reasonable alternative choices we recompute total Quality Q for all 45 (model, strategy) baseline configurations under a 3×3 grid $s_T \in \{500, 1000, 2000\}$, $s_A \in \{1, 2, 4\}$ (each scale halved, doubled, and at default), holding all other weights fixed (`scripts/d5_robustness.py`). Spearman ρ and Kendall τ against the default ranking are uniformly high (Tab. 6): $\rho \in [0.966, 1.000]$ across all 9 cells, with the top-5 set retaining at least 4 of 5 members in every cell. The most aggressive perturbation ($s_T=500$, $s_A=4$, halving the TPV scale and doubling the API scale) yields the lowest $\rho=0.966$; the default and adjacent scales yield $\rho>0.999$. The exponential form is robust to scale specification within a $2\times$ envelope; the choice (1000, 2) encodes typical streaming-API per-token-dominated pricing rather than a load-bearing modeling choice.

Table 6: D5 exponential-scale robustness: Spearman ρ and Kendall τ of the 45-cell Quality ranking against the default (s_T, s_A)=(1000, 2). Top-5 Jaccard is the overlap of the top-5 set with the default top-5.

s_T	s_A	Spearman ρ	Kendall τ	Top-1 stable	Top-5 Jaccard
500	1	0.971	0.875	no	0.667
500	2	0.970	0.877	no	0.667
500	4	0.966	0.869	no	0.667
1000	1	1.000	0.996	yes	1.000
1000	2	1.000	1.000	—	1.000
1000	4	1.000	0.994	yes	1.000
2000	1	0.988	0.925	no	1.000
2000	2	0.991	0.937	no	1.000
2000	4	0.990	0.933	no	1.000

E VGIG Pseudocode, CAAP Decision Tree, and Feedback Templates

This appendix gives the implementation-level details of the two improvement methods (VGIG and CAAP) evaluated in App. F.4. The reference implementation lives at `graphinstruct/improvements/` in the released code (`runners.py`, `caap.py`, `feedback.py`, `domain_priors.py`).

Algorithm 1: VGIG iterative refinement. For each instruction, K independent refinement chains are run (each chain at a slightly jittered temperature for diversity). Each chain does at most $T+1$ generation calls: 1 round-0 baseline plus up to T feedback-driven refinement steps. A chain terminates early when the satisfaction rate reaches 1, or when two consecutive parse failures occur.

Table 7: VGIG (`runners.VGIGRunner`). Inputs: instruction I , model M , max rounds T , num chains K , feedback level $\ell \in \{\text{none}, \text{coarse}, \text{fine}\}$, base temperature τ . Output: K refined samples.

```

1  for chain  $k = 1, \dots, K$  do
2     $\tau_k \leftarrow \text{clip}(\tau + \delta_k, [0.1, 1.5])$  // jittered round-0 temperature
3     $\hat{\tau}_k \leftarrow \max(0.2, 0.6 \tau_k)$  // lower temperature for refinement
4     $x \leftarrow M(\text{prompt}_0(I); \tau_k)$ 
5     $s \leftarrow \text{satisfaction\_rate}(x, I)$ 
6    if  $I.\text{feasible}$  is false then emit  $x$ ; continue
7     $f \leftarrow 0$  // consecutive parse-failure counter
8    for  $t = 1, \dots, T$  do
9      if  $s \geq 1.0$  then break
10     if  $\text{parse}(x)$  failed then
11        $f \leftarrow f + 1$ ; if  $f \geq 2$  then break
12        $x' \leftarrow M(\text{prompt}_0(I); \tau_k)$ ; update  $x, s$  if better; continue
13        $f \leftarrow 0$ 
14        $\text{fb} \leftarrow \text{generate\_feedback}(x, I, \ell)$ 
15       if  $\text{fb}$  is empty then break
16        $x' \leftarrow M(\text{refine\_prompt}(I, x, \text{fb}, t); \hat{\tau}_k)$ 
17        $s' \leftarrow \text{satisfaction\_rate}(x', I)$ 
18       if  $s' > s$  then  $x \leftarrow x'$ ;  $s \leftarrow s'$ 
19     end for
20     emit  $x$  as chain- $k$  output
21  end for

```

The chain temperatures use offsets $\delta_k \in \{-0.10, 0.00, +0.10, -0.05, +0.05\}$ cycling over chains (`runners.py:_CHAIN_TEMP_OFFSETS`), giving deterministic per-chain temperature schedules at any base τ . `generate_feedback` (`feedback.py`) materialises per-violation feedback at the requested granularity (Tab. 11). `refine_prompt` embeds the previous serialised graph, the feedback list, and the round counter t into a structured refinement instruction.

Algorithm 2: CAAP per-instruction strategy selection. CAAP selects a (strategy, prompt-augmentation) pair conditional on the instruction’s level and the target model’s tier. The decision rules are derived from the per-level signed strategy effects in Tab. 13 and the per-family CoT polarity in Tab. 14.

Table 8: CAAP (`caap.select_strategy`). Inputs: instruction I (with level, explicit constraints, optional domain), model name m . Output: `CAAPDecision = (strategy, extras dict, rationale)`.

```

1  tier  $\leftarrow \text{model\_tier}(m)$  //  $\in \{T1, T2\text{-}GPT, T2\text{-open}, T3\}$ 
2  return dispatch by  $I.\text{level}$  to the per-level decider in Tab. 9
3  // each decider may attach extras: checklist ( $L2$ ),
4  // formula ( $L3$ ), domain ( $L4$ ) – see Tab. 10

```

The decision table is per-(level, tier) at top level (24 cells), with three additional level-internal refinements:

- **L1 simple-vs-complex type.** If `graph_type` ∈ {tree, cycle, star, path, complete}, L1 routes to zero-shot regardless of tier (>90% baseline accuracy on simple types).
- **L2 / L3 / L5 prompt extras.** The L2 T3 decider attaches a per-constraint checklist; the L3 T3 decider attaches the relevant numerical-property formulas (only those mentioned in the explicit constraints).
- **L4 domain prior.** The L4 decider attaches the domain prior (Tab. 10) corresponding to `instruction.domain`, parameterised by `num_nodes`.

Table 9: CAAP decision rules (per-level × tier). Strategies: ZS = zero-shot, FS = few-shot, ZC = zero-CoT, FC = few-CoT. Extras attach to the prompt: `checklist`, `formula`, `domain` text. “T2-GPT” = GPT-4o / GPT-4.1 (CoT-negative family); “T2-open” = DeepSeek-V3, Llama-70B, Qwen3.5-35B (CoT-positive open-weight). The data-driven motivation for each rule is in the comment column (signed deltas from Tab. 13–14).

Level	Tier	(strategy, extras)	Rationale
L0	all	ZS	all tier deltas < 0.025
L1 <i>simple</i>	all	ZS	simple types: >90% baseline
L1 <i>complex</i>	T3	ZC	ZC safest incremental at low capability
L1 <i>complex</i>	T2-GPT	FS	FS works; GPT family hates FC
L1 <i>complex</i>	T2-open / T1	FC	FC positive (Qwen3.5 +0.052)
L2	T3	ZC + checklist	FS toxic at L2 (−0.034 avg; GPT-4o-mini drops to 0.424)
L2	T2-GPT	FS	T2-GPT tolerates FS in v4
L2	T2-open / T1	FC	FC works for non-GPT families
L3	T3	ZC + formula	FC toxic at L3 (−0.048 avg)
L3	T2-GPT	ZS	ZS neutral; FC clearly bad
L3	T2-open / T1	ZC	small ZC positive (+0.004)
L4	T3 / T2-GPT	FS + domain	FS the only +0.069 strategy at L4
L4	T2-open / T1	FC + domain	FC + domain prior best for non-GPT
L5	T3	ZC	FC toxic for T3 at L5 (−0.041)
L5	T2-GPT / T2-open / T1	FC	FC has highest signed delta (+0.045)

L4 domain priors. The CAAP L4 decider injects a structural prior describing the expected statistics of the target domain (`domain_priors.DOMAIN_PRIORS`). Eight priors cover the L4 reference-pool domains. Each prior reports an average-degree band, a clustering-coefficient band, a density band, the qualitative degree distribution, and 1–2 motif hints. The rendered prompt augmentation has the form “Domain context: {name} network (*n* nodes). Expected structural properties: ...”.

Feedback templates by level. `generate_feedback` emits a per-violation list whose richness is controlled by the level argument ℓ . Tab. 11 summarises the coarse-vs-fine difference at each instruction level. The verify-only mode (ℓ =none) emits an empty feedback string when the satisfaction check passes and a single binary “some constraint failed” marker otherwise; this is the configuration that captures 75% of VGIG’s gain (App. F.4, RQ9).

F Evaluation: Per-RQ Extended Tables, Figures, and Mechanism Details

This appendix gives the full per-RQ treatment that complements the thematic narrative of §5. Each of RQ1–RQ10 is reported under the same five-paragraph structure (*Motivation*, *Setup*, *Results*, *Mechanism*, *Implications*). Subsections F.1–F.5 group RQs by the parent thematic subsection in the main text (§5.1–§5.4), so this appendix can be read either as a per-RQ commentary or as a per-thematic deep-dive.

Table 10: L4 domain priors injected by CAAP. Bands give expected [min, max]; degree distribution is qualitative; “hint” is a single-line summary of the structural prior.

Domain	Avg deg.	Clustering	Density	Deg. dist.	Hint
social	4.0–6.0	0.40–0.70	0.30–0.50	power-law	friend circles + hubs
citation	2.5–4.0	0.05–0.25	0.05–0.15	power-law	DAG; seminal-paper hubs
biological	2.0–5.0	0.10–0.40	0.10–0.30	power-law	sparse with functional modules
ecological	2.0–4.0	0.15–0.35	0.10–0.25	roughly-uniform	shallow producer-consumer hierarchy
communication	3.0–8.0	0.20–0.50	0.30–0.50	power-law	active hubs, bursty interactions
infrastructure	2.0–4.0	0.05–0.20	0.05–0.20	roughly-uniform	grid-like; high-betweenness backbone
knowledge_graph	2.0–6.0	0.05–0.30	0.05–0.20	power-law	typed relations; broad-concept hubs
molecular	1.5–3.5	0.00–0.15	0.05–0.25	roughly-uniform	valence-limited ($C \leq 4$, $N \leq 3$, $O \leq 2$)

Table 11: VGIG feedback templates by level. *Coarse* reports violation category; *Fine* reports per-constraint (expected, observed, Δ). Template richness is level-specific because constraint types differ structurally.

Level	Coarse	Fine (additions)
L0, L1	{violation_ids}	+ {expected, observed} format/value details
L2	{constraint-category conflicts}	+ Δ per constraint and joint-incompatibility flags
L3	{attribute deficits}	+ numeric deltas, tolerance band
L4	{missing motifs}	+ domain-baseline comparison
L5	{failed ops}	+ step-wise partial credit, intermediate state diff

E.1 RQ1 & RQ2: Capability Stratification

RQ1: Where in the complexity spectrum does LLM graph-generation capability most sharply differentiate?

Motivation. A diagnostic benchmark must separate models with different capabilities. Aggregate benchmarks deliver that separation at the task level; we ask where the separation lies *within* a single task family once outputs are stratified by complexity.

Setup. For each of the 6 levels, we compute the mean Quality per capability tier (T1/T2/T3, §3.1), averaged over all 45 (model, strategy) configurations within each tier (T1: 3 models $\times$ 4 strategies = 12 cells; T2: 5 models $\times$ 4 + Sonnet-4 $\times$ 1 = 21 cells; T3: 3 $\times$ 4 = 12 cells), and define the tier gap as $Q_{T1} - Q_{T3}$.

Results. Table 12 shows the per-level mean by tier; Figure 3 renders the same data as a bar chart.

Table 12: Per-level mean Quality by tier and T1–T3 gap.

Level	T1 mean	T2 mean	T3 mean	T1–T3 gap
L0	0.941	0.925	0.884	0.057
L1	0.975	0.965	0.855	0.120
L2	0.917	0.862	0.698	0.219
L3	0.858	0.815	0.785	0.073
L4	0.841	0.800	0.719	0.122
L5	0.879	0.835	0.773	0.106

Mechanism. One might expect the reasoning-heavy levels—L3, L4, or L5—to produce the largest tier gap, because these levels involve longer causal chains from instruction to output. The data

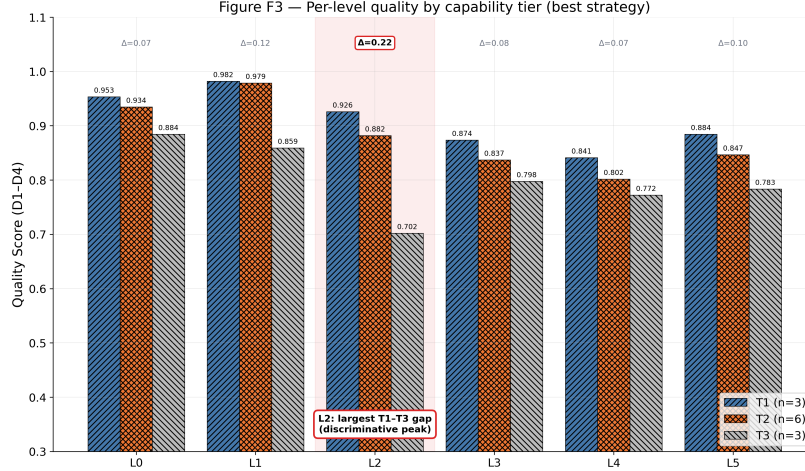


Figure 3: Per-level Quality by capability tier, averaged over the 45 (model, strategy) configurations in the capability evaluation. The T1–T3 gap at L2 (0.219) is $3\times$ the next-widest reasoning-heavy gap (L3, 0.073) and $2\times$ the multi-step-reasoning gap (L5, 0.106), localizing benchmark discriminative power at multi-constraint composition rather than reasoning depth.

contradict this: **L2, simultaneous satisfaction of four or more structural constraints, is where the tier gap is largest.** The most plausible mechanism is compositional—each added constraint multiplies the probability of violation, and T3 models lack the working-memory capacity to maintain joint satisfaction.

L2 failure is not merely harder, it is more brittle. Per-instruction D1 variance at L2 (averaged over the 10 zero-shot-evaluated models) is $\sigma = 0.240$, roughly $2.2\times$ the L3 value (0.111) and $2.4\times$ the L4 value (0.101); see Figure 4. Weak models are the worst offenders: GPT-4o-mini and GPT-3.5 reach $\sigma_{L2} > 0.43$, while Sonnet-4.6 and Qwen3.5-397B stay at $\sigma_{L2} < 0.13$. This means L2 is a *compositional instability* regime, not a uniformly harder regime: weak models succeed brilliantly on some L2 instructions and fail catastrophically on others, depending on which particular constraint combination is sampled. This is the mechanism behind the $3\times$ tier gap—T3 models are one constraint-combination away from collapse at L2, while T1 models are robust to the same combinations—and it directly motivates reporting *per-constraint-type* method gains on L2 rather than level-wise averages.

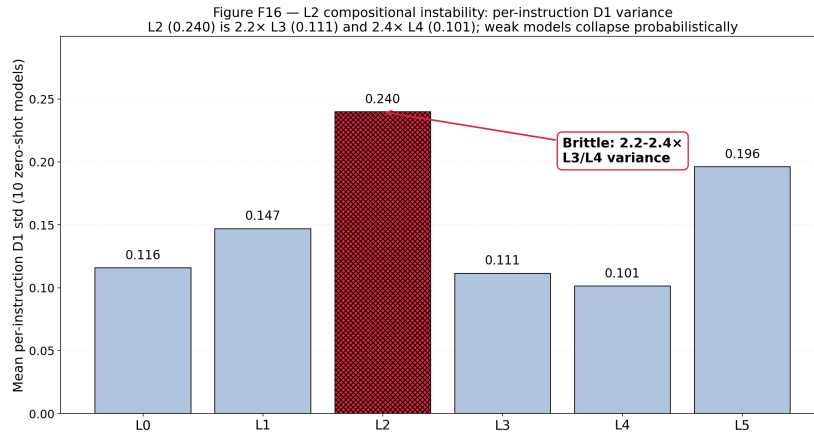


Figure 4: Per-instruction D1 standard deviation by level, averaged over 10 zero-shot models. L2 ($\sigma=0.240$) is $2.2\times$ L3 and $2.4\times$ L4, reflecting compositional instability rather than uniform difficulty.

Case study. Figure 5 renders the L2-143 instruction (“Generate a 3×5 grid graph with 15 nodes that is connected and planar”) side-by-side for the algorithmic reference, Sonnet-4.6 (T1), and GPT-

4o-mini (T3). Both Sonnet-4.6 and the reference produce a clean grid that satisfies every constraint; GPT-4o-mini emits a 19-node, 26-edge graph that violates four of seven constraints (graph type, node count, edge count, minimum degree). The qualitative picture matches the quantitative finding: at L2, frontier models hold the spec while small models drift in node count and lose the joint grid structure.

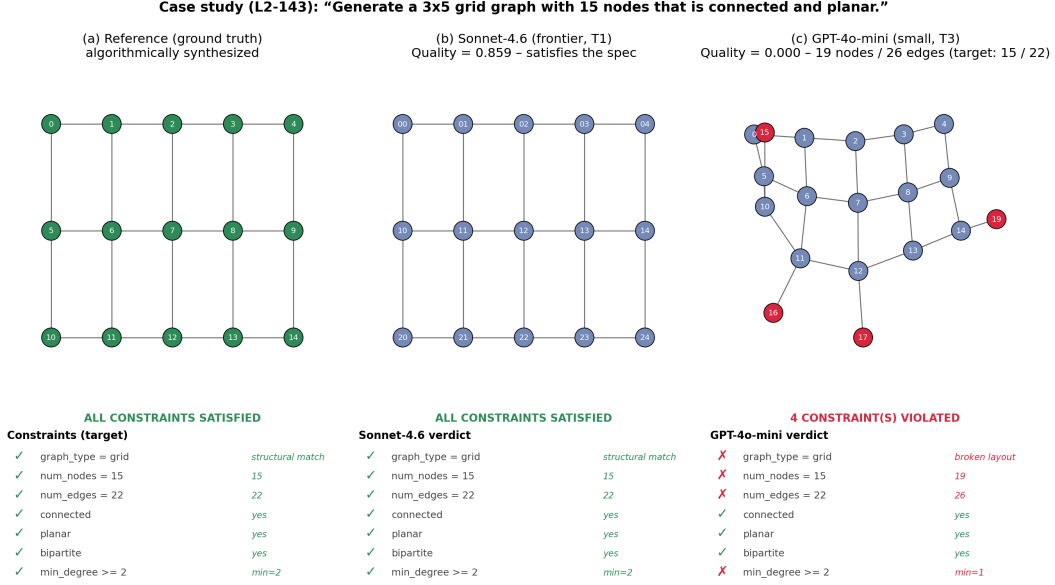


Figure 5: Capability-gap case study at L2 (instruction L2-143). Reference (left) and Sonnet-4.6 (centre) both produce a 15-node 3×5 grid satisfying all seven constraints. GPT-4o-mini (right) emits a parseable but constraint-violating 19-node graph (red highlighted nodes are the four extras: 15, 16, 17, 19), failing four of seven constraints. The constraint checklists below each panel make the failure modes machine-checkable.

Implications. Benchmarks averaging over constraint count *systematically underestimate* structured-generation discriminative power. Per instruction, L2 is 3× more informative than aggregate scoring. Method research targeting structural-generation improvement should report L2 gains as a primary signal, not overall score. Expanding L2 in future versions of GraphInstruct is a natural way to sharpen this signal further.

RQ2: How does prompt-strategy sensitivity vary with base capability?

Motivation. Demirci et al. (2025) report that iterative-feedback gains vary substantially across models (Grok +48%, Llama < 5%) without identifying an explanatory variable. We test whether this heterogeneity is predictable from base capability.

Setup. For each model we compute σ_{strat} , the standard deviation of Q across the four prompting strategies. Because σ_{strat} requires four-strategy data, Sonnet-4 is excluded from this analysis (zero-shot-only, §3.1); we regress σ_{strat} on mean Q across the remaining **11 fully-evaluated models**.

Results. Strategy variance varies sharply with capability: T3 models exhibit $\sigma_{\text{strat}} \in \{0.074, 0.070, 0.048\}$ (GPT-3.5, GPT-4o-mini, Llama-8B respectively); T2-stable models exhibit $\sigma_{\text{strat}} \in \{0.019, 0.018\}$ (DeepSeek-V3, Llama-70B); T1 models exhibit $\sigma_{\text{strat}} \in \{0.043, 0.050, 0.040\}$. OLS regression of σ_{strat} on mean Q yields $\beta = -0.27$, $R^2 = 0.62$, $p < 10^{-3}$.

Mechanism. Weak models occupy an under-trained region of the output manifold where small prompt perturbations—adding a CoT trigger, switching from zero-shot to few-shot—produce large quality displacements. Strong models occupy a better-conditioned region with a flatter local quality surface.

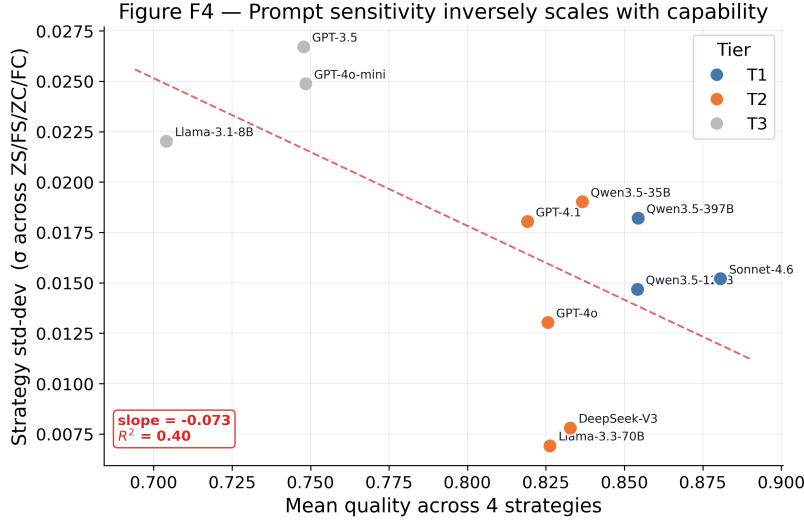


Figure 6: Prompt sensitivity (σ_{strat} , y-axis) vs. base capability (mean Q , x-axis) across the 11 fully-evaluated models (Sonnet-4 excluded, zero-shot-only). The $4\times$ gap between weakest T3 models ($\sigma_{\text{strat}} = 0.074$) and most prompt-stable T2 models ($\sigma_{\text{strat}} = 0.019$) establishes an inverse-scaling relation; the solid line is an OLS fit ($R^2 = 0.62$).

Implications. Prompt-engineering budgets should scale inversely with model capability: for frontier models, strategy choice accounts for $\leq 2\%$ of performance variance, so investing in verification or retrieval yields better returns. Single-strategy benchmark evaluations systematically disadvantage prompt-sensitive models. Prompting-method papers should report per-model σ_{strat} alongside headline gains.

F.2 RQ3, RQ4 & RQ5: Strategy and Family Effects

RQ3: Does any single prompting strategy uniformly dominate across complexity levels?

Motivation. Aggregate benchmark scores wash out level-dependent strategy effects. We test whether such effects exist and whether they change sign across levels.

Setup. We compute per-level strategy effects relative to zero-shot (FS–ZS, ZC–ZS, FC–ZS), averaged across the **11 fully-evaluated models** (Sonnet-4 excluded because the three non-zero-shot strategies are undefined for it).

Results. Table 13 shows the signed effects; Figure 7 renders the same data as a heatmap.

Table 13: Signed strategy–level effects (averaged across the 11 fully-evaluated models), Δ vs. zero-shot.

Level	FS – ZS	ZC – ZS	FC – ZS	Best strategy
L0	−0.016	+0.007	−0.025	zero-CoT
L1	−0.024	+0.008	−0.016	zero-CoT
L2	−0.034	+0.038	−0.018	zero-CoT
L3	+0.003	+0.004	−0.048	zero-CoT
L4	+0.069	−0.007	+0.049	few-shot
L5	+0.003	+0.037	+0.045	few-CoT

Mechanism. No strategy dominates: every non-trivial strategy is *net-harmful* at at least one level and *net-helpful* at at least one other. Few-shot is poison at L2—the demonstration’s specific graph biases generation toward copying topology rather than satisfying the target’s distinct constraints—but

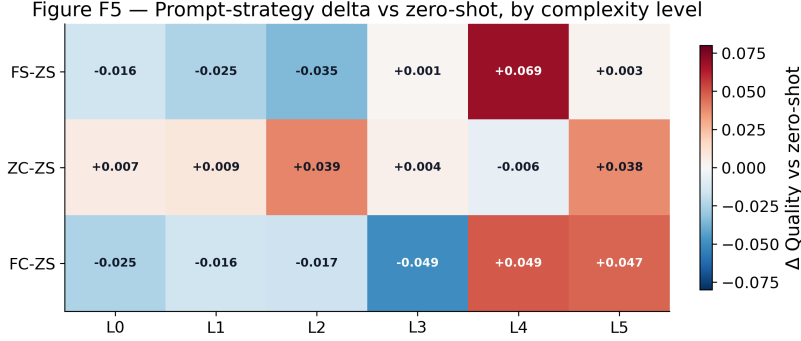


Figure 7: Signed strategy $\times$ level effect heatmap (average over the 11 fully-evaluated models). Few-shot is net-negative at L2 (-0.034) and net-positive at L4 ($+0.069$); few-CoT swings from net-negative at L3 (-0.048) to net-positive at L5 ($+0.045$). Aggregate benchmarks mask these opposite-signed effects.

savior at L4, where domain examples convey structural priors the instruction alone cannot. Few-CoT is savior at L5 (explicit step planning for edits) but actively harmful at L3 (extraneous reasoning amplifies numerical drift). Zero-CoT is the only strategy with *non-negative effect at every level*, though its gains are modest where other strategies excel.

Implications. Prompting-method papers reporting only aggregate gains may be silently trading L2 for L4 or L3 for L5. Level-stratified reporting should become standard on structured tasks. Strategy-per-level adaptive prompting is empirically motivated, directly informing our CAAP component (§4.3).

RQ4: Does chain-of-thought transfer uniformly across model families?

Motivation. The folk wisdom that “CoT helps on complex tasks” rests almost entirely on text-domain benchmarks. We test whether the transfer holds on graph generation and whether it is uniform across model families.

Setup. We compute CoT effects (ZC–ZS and FC–ZS) for the seven family-aligned models we evaluate: three Qwen3.5 scales and four GPT-family scales.

Results. Table 14 reveals a sign reversal across families; Figure 8 visualizes the family-level polarity.

Table 14: CoT deltas (vs. zero-shot) by model family and scale.

Family	Model	ZC – ZS	FC – ZS
Qwen3.5	35B-A3B	+0.037	+0.052
Qwen3.5	122B-A10B	+0.029	+0.040
Qwen3.5	397B-A17B	+0.032	+0.050
GPT	3.5-turbo	+0.032	−0.042
GPT	4o-mini	+0.032	−0.038
GPT	4o	−0.020	−0.005
GPT	4.1	−0.010	−0.002

Mechanism. Few-CoT is *uniformly beneficial* for Qwen3.5 across scales (35B, 122B, 397B) and *uniformly detrimental* for GPT-family across scales (3.5, 4o-mini, 4o, 4.1). The sign’s robustness across scales within each family—GPT-4.1 agrees with GPT-3.5 despite spanning roughly two orders of magnitude of capability; Qwen3.5-35B agrees with Qwen3.5-397B despite a $\sim 10\times$ parameter gap—indicates CoT effectiveness is governed by **pretraining distribution** rather than capability tier. We conjecture that Qwen3.5’s heavy inclusion of code, mathematical, and reasoning-chain

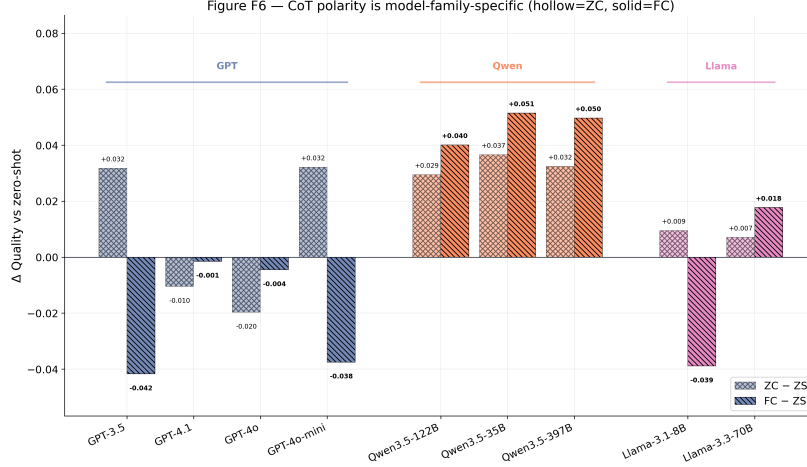


Figure 8: Signed CoT effect by model family. Qwen3.5 gains uniformly across scales (+0.029 to +0.052); GPT-family few-CoT is net-negative at every observed scale (−0.042 to −0.002). The sign is robust across scales within each family.

data creates a CoT prior that transfers positively to graph generation, while GPT-family models’ broader but less code-heavy pretraining diet creates a weaker or negatively-transferring prior. Direct verification would require pretraining-mixture ablations beyond our scope.

Implications. The text-domain folk wisdom surrounding chain-of-thought (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023b) does not transfer to graph generation for one major model family. Prompting-method claims derived from GPT evaluations cannot be generalized without independent validation on other families. Model-family should be reported alongside capability when benchmarking CoT-using methods.

RQ5: Does parameter scaling uniformly translate into per-level capability?

Motivation. Aggregate scaling-law narratives predict monotone improvement with parameter count. Progressive evaluation lets us test that prediction *per-level* within a same-family scale series.

Setup. We compare same-family models across scales: Qwen3.5 at 35B/122B/397B, and GPT-family where comparable at 3.5/4o-mini/4o/4.1. We report both aggregate and per-level Quality.

Results. Parameter scaling does not uniformly lift per-level capability (Table 15); per-level Qwen3.5 scaling curves are in Figure 9.

Table 15: Scale decoupling: cases where a larger model fails to outperform (or underperforms) a smaller same-family model.

Subtask	Smaller	Larger	Δ
L3 numerical attributes	Qwen3.5-35B, 0.833	Qwen3.5-397B, 0.865	+0.032
L4 domain semantics	Qwen3.5-35B, 0.818	Qwen3.5-397B, 0.842	+0.024
L5 graph editing	Qwen3.5-35B, 0.877	Qwen3.5-397B, 0.872	−0.005
L3 numerical attributes	GPT-3.5, 0.808	GPT-4.1, 0.759 (avg)	−0.049

Mechanism. Tasks that reduce to **local structural operations**—bounded-scope editing (L5), single-attribute verification (parts of L3)—saturate at smaller model scales than tasks requiring global constraint reasoning. In Qwen3.5, 35B→397B adds only 0.024 at L4 and *subtracts* 0.005 at L5, while adding 0.032 at L3. The GPT counter-example is more striking: GPT-3.5 *outperforms* GPT-4.1 on L3 numerical attributes (0.808 > 0.759 avg), a reversal aggregate quality conceals entirely. The mechanism interacts with RQ4: larger GPT-family models’ CoT inclinations produce longer, more error-prone reasoning chains on tasks smaller models attempt more directly.

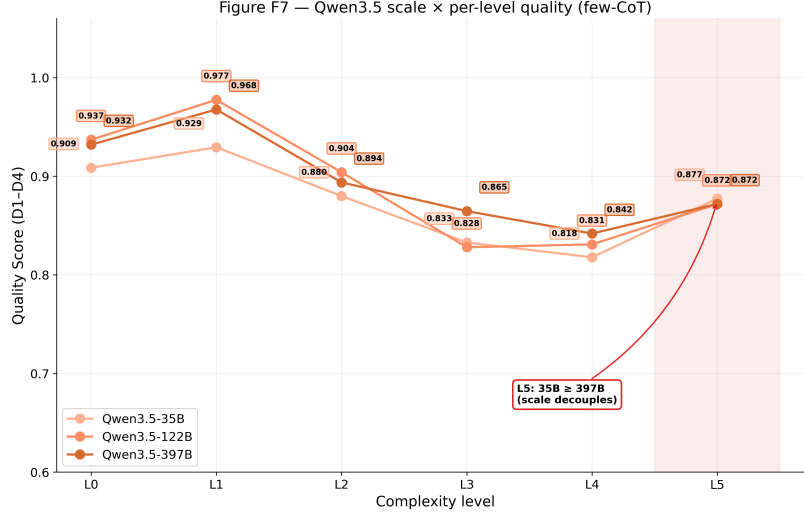


Figure 9: Qwen3.5 scale family (35B / 122B / 397B) per-level Quality. Scaling monotonically improves L0–L4 but saturates at L5, where 35B matches 397B within ± 0.005 .

Implications. Scaling-law studies (Kaplan et al., 2020; Hoffmann et al., 2022) built on aggregate benchmarks mask task-structural heterogeneity in where capability emerges with scale, extending the emergent-behavior critique of Schaeffer et al. (2023) to a specific structured-generation sub-capability. Parameter-efficient deployment strategies—“smaller model for subtask A, larger model for subtask B”—are viable on structured tasks in a way aggregate benchmarks obscure.

F.3 RQ6: Cost-Aware Deployment

RQ6: How does cost-adjusted scoring reshape model rankings, and where is the capability floor?

Motivation. RQ1–RQ5 characterize capability on Quality S_{total} . Deployment practitioners also care about cost: a $+3\%$ gain at $5\times$ token cost is net-negative if a cheaper alternative already clears the usable-quality threshold. We apply the benchmark’s efficiency instruments—the Pareto-adjusted S_{final} (Eq. 3), $Q/k\text{TPV}$, and $\text{Cost}@Q=0.8$ —to the 45 baseline configurations, treating S_{final} as a *complementary* deployment-oriented view alongside the capability-characterization view given by S_{total} .

Setup. For each of the 45 runs we compute mean tokens per valid graph (TPV), Quality S_{total} , $Q/k\text{TPV}$, the non-dominated Pareto frontier in $\langle \text{TPV}, S_{\text{total}} \rangle$ space, and per-model $\text{Cost}@Q=0.8$. We then compute S_{final} with $\lambda = 0.15$ and compare the S_{total} and S_{final} orderings. Per-model Pareto frontiers in $(S_{\text{total}}, 1/D_5)$ space, with annotations for the four prompting strategies, are drawn from the same data and follow the per-configuration trajectory shown here.

Result (a): A 6-point, provider-concentrated Pareto frontier. Only **6 of 45** configurations are non-dominated (Table 16; visualized in Figure 10), all Anthropic or OpenAI; no Qwen3.5, DeepSeek, or Llama configuration is Pareto-optimal despite several near-frontier candidates (e.g., Qwen3.5-397B zero-CoT reaches $Q=0.862$ at $\text{TPV}\approx 2300$ but is dominated by Sonnet-4.6 zero-CoT at $\text{TPV}=969$, $Q=0.878$). The frontier follows a strategy progression: zero-shot at the low-cost end, zero-CoT / few-shot in the middle, few-CoT on Sonnet-4.6 at the high-cost end.

Result (b): S_{final} reshuffles the leaderboard; few-CoT is absent from the efficiency top-15. The Pareto bonus moves 6 configurations up by 5–18 positions and pushes high-cost Qwen3.5 runs down by 2–4 (Table 17). The largest climb is GPT-4o zero-shot (S_{total} #24 $\rightarrow$ S_{final} #6, $+18$); the largest fall is Qwen3.5-397B few-CoT (#3 $\rightarrow$ #7). Among the **11 models for which per-model strategy comparison is defined**, the transition flips the S_{total} -best strategy for **only GPT-4o** (few-shot,

Table 16: The 6-point Pareto frontier over the 45 baseline configurations, ordered by ascending TPV.

Model	Strategy	mean TPV	S_{total}	$S_{\text{final}} (\lambda=0.15)$
GPT-4o	zero-shot	578	0.8274	0.9515
Sonnet-4	zero-shot	625	0.8342	0.9594
Sonnet-4.6	zero-shot	658	0.8591	0.9880
Sonnet-4.6	zero-CoT	969	0.8780	1.0097
Sonnet-4.6	few-shot	2415	0.8836	1.0161
Sonnet-4.6	few-CoT	2846	0.9018	1.0371

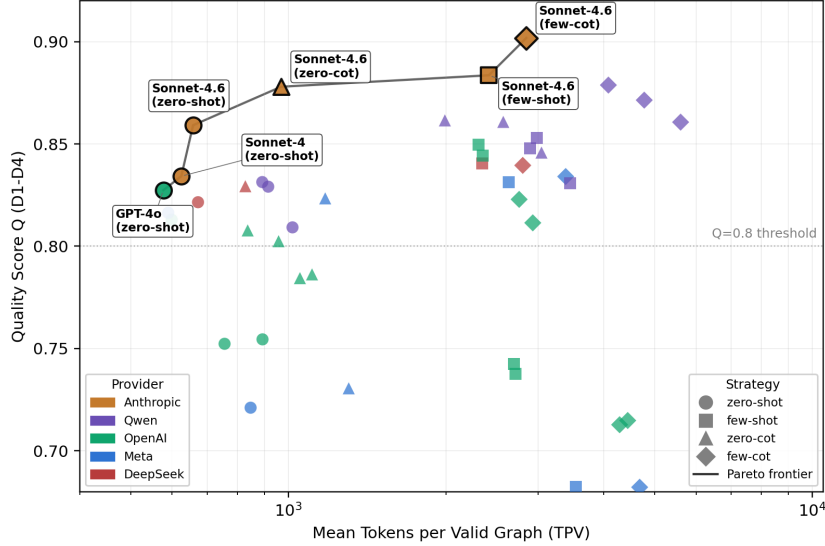


Figure 10: Pareto frontier over 45 baseline (model, strategy) configurations in $\langle \text{mean TPV}, \text{Quality} \rangle$ space. Solid line traces the 6 non-dominated points (labeled). Markers encode prompting strategy; colors encode provider. Nine of 12 models cross the $Q=0.8$ threshold (dotted line)—zero-shot is the cheapest threshold-crossing strategy for all eight that are evaluated on all four strategies (Sonnet-4 was evaluated only under zero-shot, which also clears the threshold); three T3 models never cross it.

$S_{\text{total}}=0.844 \rightarrow \text{zero-shot}$, $S_{\text{final}}=0.952$); the other 10 retain their S_{total} -optimal choice (per-model strategy comparison is undefined for Sonnet-4 since only zero-shot was evaluated). On the auxiliary Q/kTPV ranking, the top five are **all zero-shot** (GPT-4o 1.43, Llama-3.3-70B 1.38, GPT-4.1 1.36, Sonnet-4 1.34, Sonnet-4.6 1.31), and **no few-CoT configuration enters the top 15** even though few-CoT is the S_{total} -optimal strategy for 5 of the 11 fully-evaluated models.

Frontier Distance headroom. FD ranges from 0 (the six Pareto-optimal configurations) to 0.22 (Llama-3.1-8B few-CoT), with median 0.062 over the 39 off-frontier configurations (Figure 11). Two off-frontier points are *near-Pareto* at $\text{FD} < 0.02$: Llama-3.3-70B zero-shot (0.013) and GPT-4.1 zero-shot (0.018)—both are zero-shot configurations dominated only by a tighter-cost Sonnet-4.6 run, and indicate that two additional providers are a hair’s-breadth from joining the frontier. The five largest FD values all combine a T3 model with few-CoT or few-shot (Llama-3.1-8B fc/fs: 0.219/0.219, GPT-3.5 fc: 0.189, GPT-4o-mini fc: 0.187, GPT-3.5 fs: 0.159), confirming that weak-capability models *compound* their quality deficit under expensive prompting. FD is a practitioner-facing diagnostic: at a fixed cost budget it answers “how much quality am I leaving on the table by using this configuration?”

Result (c): A hard capability floor at $\text{Cost}@Q=0.8$. Table 18 reports the $\text{Cost}@Q=0.8$ matrix over the 11 models evaluated on all four prompting strategies. Eight of these 11 cross $Q \geq 0.8$ under at least one strategy, and **in every row zero-shot is the cheapest threshold-crossing strategy**. The three cheapest fully-evaluated zero-shot paths come from structurally different T2 providers: GPT-4o (TPV=578), Llama-3.3-70B (590), and GPT-4.1 (599); Sonnet-4’s zero-shot-only TPV (625) falls

Table 17: Top-15 S_{final} leaderboard with $S_{\text{total}} \rightarrow S_{\text{final}}$ rank change Δ . “Par.” marks Pareto-optimal configurations. All five $Q/k\text{TPV}$ -top rows and every rank-gainer are zero-shot; the $S_{\text{total-to-}}S_{\text{final}}$ demotions concentrate in the Qwen3.5 family. Ranks are computed across all 45 (model, strategy) configurations. † Sonnet-4 was evaluated only under zero-shot (§3.1); its rank reflects this single configuration’s global position and not a per-model best-of-four.

$S_{\text{final}} \#$	$S_{\text{total}} \#$	Δ	Model	Strategy	S_{total}	TPV	$Q/k\text{TPV}$	S_{final}	Par.
1	1	0	Sonnet-4.6	few-CoT	0.902	2846	0.32	1.037	✓
2	2	0	Sonnet-4.6	few-shot	0.884	2415	0.37	1.016	✓
3	4	+1	Sonnet-4.6	zero-CoT	0.878	969	0.91	1.010	✓
4	9	+5	Sonnet-4.6	zero-shot	0.859	658	1.31	0.988	✓
5	17	+12	Sonnet-4†	zero-shot	0.834	625	1.34	0.959	✓
6	24	+18	GPT-4o	zero-shot	0.827	578	1.43	0.952	✓
7	3	−4	Qwen3.5-397B	few-CoT	0.879	4082	0.22	0.879	
8	5	−3	Qwen3.5-122B	few-CoT	0.871	4779	0.18	0.871	
9	6	−3	Qwen3.5-397B	zero-CoT	0.862	1990	0.43	0.862	
10	7	−3	Qwen3.5-122B	zero-CoT	0.861	2572	0.33	0.861	
11	8	−3	Qwen3.5-35B	few-CoT	0.861	5607	0.15	0.861	
12	10	−2	Qwen3.5-122B	few-shot	0.853	2980	0.29	0.853	
13	11	−2	GPT-4.1	few-shot	0.850	2308	0.37	0.850	
14	12	−2	Qwen3.5-397B	few-shot	0.848	2894	0.29	0.848	
15	13	−2	Qwen3.5-35B	zero-CoT	0.846	3042	0.28	0.846	

in the same narrow band but is a single-strategy observation (§3.1) and therefore excluded from the 4-strategy matrix. Three T3 models—**GPT-3.5**, **GPT-4o-mini**, **Llama-3.1-8B**, each evaluated on all four strategies—**never** reach $Q \geq 0.8$ under any, marking an empirical capability floor that prompting alone cannot cross and directly motivating the verification-guided methods in RQ7–RQ10.

Table 18: Cost@ $Q=0.8$ matrix over the 11 models evaluated on all four prompting strategies. Each cell shows the mean TPV to reach $S_{\text{total}} \geq 0.8$ for that (model, strategy), or “—” if $S_{\text{total}} < 0.8$. Bold marks the per-model cheapest threshold-crossing strategy—zero-shot in every row. Three T3 models never cross the threshold under any strategy. **Sonnet-4 is excluded from this matrix** because only zero-shot was evaluated in the baseline survey (§3.1); for reference, Sonnet-4 zero-shot reaches $Q=0.834$ at TPV=625, which also clears the threshold. Including Sonnet-4 zero-shot brings the threshold-crossing count to 9 of 12 models.

Model	Tier	zero-shot	few-shot	zero-CoT	few-CoT	best
Sonnet-4.6	T1	658	2415	969	2846	658
Qwen3.5-122B	T1	892	2980	2572	4779	892
Qwen3.5-397B	T1	916	2894	1990	4082	916
GPT-4o	T2	578	2350	837	2758	578
Llama-3.3-70B	T2	590	2636	1176	3384	590
GPT-4.1	T2	599	2308	958	2928	599
DeepSeek-V3	T2	673	2344	828	2802	673
Qwen3.5-35B	T2	1019	3453	3042	5607	1019
GPT-3.5	T3	—	—	—	—	<i>never</i>
GPT-4o-mini	T3	—	—	—	—	<i>never</i>
Llama-3.1-8B	T3	—	—	—	—	<i>never</i>

Mechanism. Sonnet-4.6’s zero-shot configuration (TPV=658, $Q=0.859$) already dominates every fully-evaluated T2 model’s most expensive strategy (few-CoT, TPV>2700) on both axes—so the high-cost end of the frontier is an intra-Sonnet-4.6 cost–quality trade, not a cross-model one. The $3.3\times$ TPV increase from Sonnet-4.6 zero-shot to few-CoT buys only +0.043 Quality, foreshadowing the ~ 5 -round refinement saturation we document in RQ10. CoT’s token overhead without a compensating quality gain on GPT-family (RQ4) keeps every GPT-family CoT configuration off the frontier. Within the 8 threshold-crossers evaluated on all four strategies, every model’s cheapest path is zero-shot, localizing the cost–quality trade-off to a single dimension: prompting-strategy-induced token inflation.

Implications. Quality-only benchmark rankings understate the efficiency advantage of zero-shot baselines for efficiency-sensitive deployment. We therefore recommend reporting S_{final} as a *complementary* deployment-oriented leaderboard alongside, not replacing, S_{total} : the two answer different

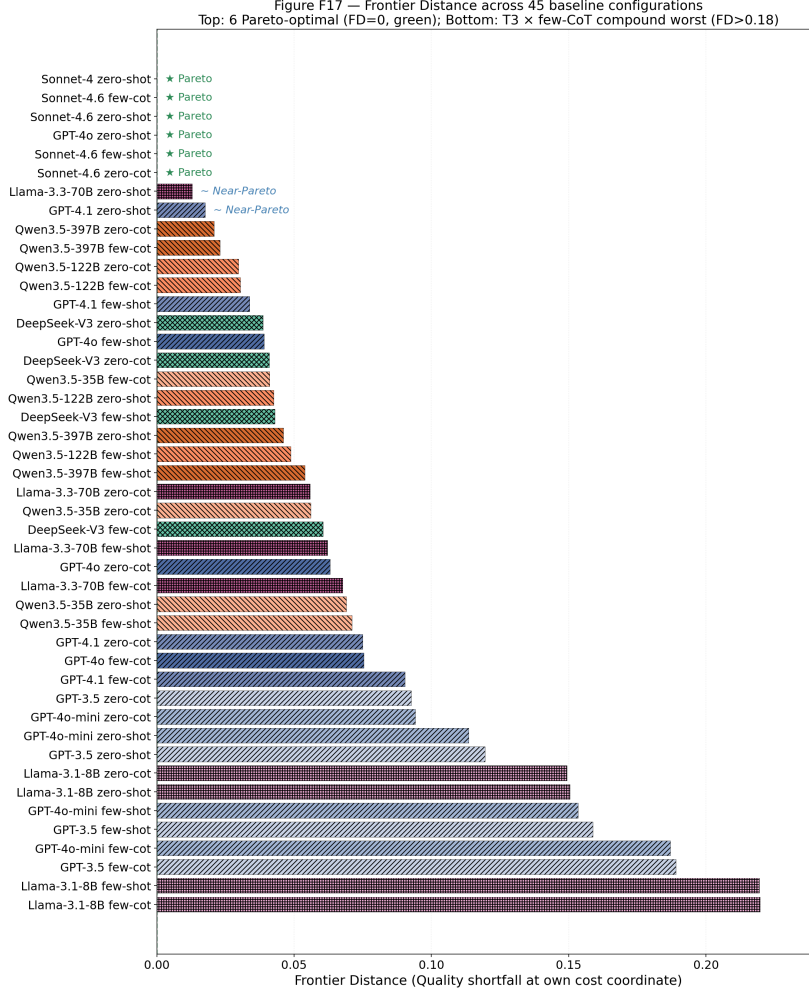


Figure 11: Frontier Distance across 45 baseline configurations, sorted ascending. Top: 6 Pareto-optimal (FD=0, green star); near-Pareto zero-shot runs (Llama-3.3-70B, GPT-4.1) follow. Bottom: T3 × few-CoT compounds quality deficit under expensive prompting.

questions. The TPV ≈ 600 floor across 9 of 12 models provides a calibrated budget anchor for future efficiency-aware comparisons, and the 3-model T3 shortfall defines a concrete capability target for the method research of RQ7–RQ10.

F.4 RQ7–RQ10: Methods Atop Benchmark Signals (extended)

RQ7: Can fine-grained benchmark signals drive method improvement beyond the prompt-engineering ceiling?

Motivation. RQ1–RQ5 establish an empirical ceiling on prompt engineering: no single strategy wins across levels (RQ3) and prompt sensitivity saturates at strong capability (RQ2). We test whether the benchmark’s fine-grained D4 signal—unavailable to prompt-only methods—can push past this ceiling.

Setup. We define **Oracle** as the per-level best-of-four selection over the four prompting strategies, evaluated per model. Oracle is an empirical upper bound on any prompt-only method because it requires oracle knowledge of the optimal per-level strategy. We compare against CAAP-only (learned strategy selection), VGIG-only (verification-guided iteration), and Combined on three target models spanning capability tiers.

Results. Combined exceeds Oracle by +0.035 to +0.050 on every tested model (Table 19, Figure 12).

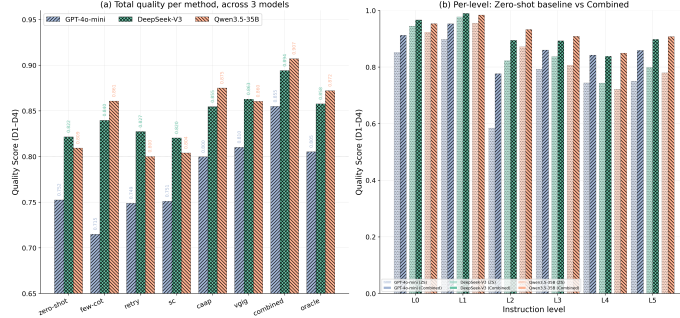


Figure 12: Method $\times$ model Quality with per-model Oracle reference line. Combined surpasses Oracle by +0.035–+0.050 on every target model; VGIG-only contributes the majority of the gain.

Table 19: Quality Score for the three target models across main method conditions. Combined clears Oracle by +0.035–+0.050 on every model.

Model	ZS	Oracle	VGIG	CAAP	Combined	Δ vs. Oracle
GPT-4o-mini	0.7523	0.8052	0.8201	0.8128	0.8549	+0.050
DeepSeek-V3	0.8215	0.8577	0.8743	0.8629	0.8941	+0.036
Qwen3.5-35B-A3B	0.8092	0.8720	0.8833	0.8755	0.9071	+0.035

Mechanism. Prompt engineering has a measurable empirical ceiling; external programmatic verification—not prompt phrasing—is the binding mechanism for reliable structured graph generation. The margin is robust: +0.035 is $7\times$ the ± 0.005 noise band (RQ10). The gap is *largest* for the weakest target (GPT-4o-mini, +0.050) and *smallest* for the strongest (Qwen3.5-35B, +0.035), mirroring RQ2: the low-capability regime leaves the most room for method intervention.

Implications. A benchmark that exposes fine-grained per-constraint verification signals (our D4) is not just a better evaluation instrument but a *development platform* enabling method families that aggregate or pass/fail benchmarks cannot support.

RQ8: Is sampling compute alone, without verification, sufficient to improve structured generation?

Motivation. A natural alternative hypothesis to RQ7 attributes the gain to *additional sampling compute* rather than verification. We test that hypothesis by evaluating pure retry (additional samples without feedback) and self-consistency (best-of- N) on the same three target models.

Setup. Retry samples $T=3$ candidates without feedback and returns the last; self-consistency (SC) samples $N=3$ independent candidates and returns the majority/best. Both are compared against the zero-shot baseline.

Results. Both stagnate within ± 0.01 of zero-shot on all three target models (Table 20). This is a strong null result: six controls (3 models $\times$ 2 methods) all in-band.

Mechanism. Additional sampling compute, without a verification signal to select or guide candidates, cannot recover from constraint violations on structured graph generation. This contrasts sharply with text-domain results where self-consistency (Wang et al., 2023b) offers meaningful gains (e.g., 5–10% on GSM8K (Cobbe et al., 2021)). The difference is structural: text tasks have many plausible-looking outputs of which only some are correct (majority-vote recovers); structured-graph tasks have many wrong outputs with violating structures (majority-voting among wrong outputs does not help).

Table 20: Retry and SC vs. zero-shot on target models. All six controls fall within the ± 0.01 noise band.

Model	ZS	retry	SC	retry Δ	SC Δ
GPT-4o-mini	0.7523	0.7488	0.7510	-0.003	-0.001
DeepSeek-V3	0.8215	0.8273	0.8203	+0.005	-0.001
Qwen3.5-35B	0.8092	0.8001	0.8039	-0.009	-0.005

Implications. Verifiable structured tasks have a fundamentally different compute-to-quality relation than text tasks. Deployment budgets should be allocated to verification infrastructure, not parallel sampling.

RQ9: How does feedback granularity compare with iteration count as a refinement lever?

Motivation. RQ7 establishes that verification-guided iteration helps, and RQ8 rules out sampling-only explanations. The remaining question is which *component* of VGIG drives the gain—iteration count or feedback granularity.

Setup. E6 ablation on GPT-4o-mini at fixed $T = 3$, varying only the feedback granularity in $\{\text{retry}, \text{none}, \text{coarse}, \text{fine}\}$: *retry* has no verification at all; *none* runs verification but returns only pass/fail; *coarse* reports which constraint category failed; *fine* reports per-constraint id, expected, observed.

Results. The verify-only signal captures 75% of the total gain (Table 21, Figure 13).

Table 21: E6 feedback-granularity ablation (GPT-4o-mini, $T=3$).

Feedback config	Quality	Δ vs. previous
retry (no verify)	0.7488	—
verify-only (fb=none)	0.7946	+0.046
coarse (fb=coarse)	0.8021	+0.008
fine (fb=fine)	0.8099	+0.008

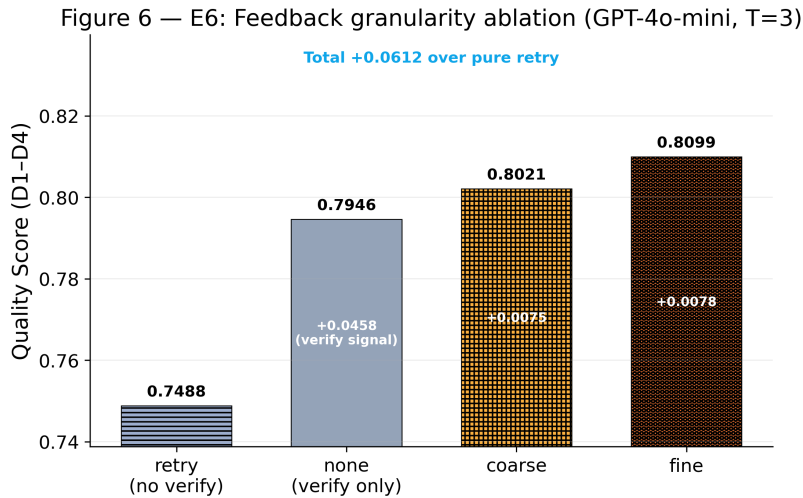


Figure 13: E6 feedback-granularity ablation on GPT-4o-mini at $T=3$. Verify-only captures 75% of total gain; coarse and fine each add +0.008 monotonically. The ordering $\text{retry} < \text{none} < \text{coarse} < \text{fine}$ holds across all six levels.

Mechanism. The verify-only signal alone—binary pass/fail, no constraint-level detail—captures 75% of the total gain (+0.046 of +0.061); coarse and fine each contribute a further +0.008, monotonically. The richness of the error signal, not the compute budget, is the primary lever for iterative refinement on structured tasks.

Implications. This directly informs benchmark design: a benchmark that exposes even a coarse verify-true/verify-false signal enables verification-driven method research; our fine per-constraint signal unlocks the additional 25%. Benchmarks that lack a programmatic verification signal are missing not only an evaluation dimension but also a method-enabling resource for the community that will build atop them.

RQ10: How far can iterative refinement go, and does any failure mode resist it entirely?

Motivation. Two linked questions: (a) what is the effective refinement horizon on verifiable graph constraints, and (b) **does any failure mode resist iteration entirely, regardless of round count or feedback richness?** An iteration-invariant failure mode would redirect the research program from compute scaling (more rounds, richer feedback) to capability-structural interventions (retrieval, grounding), so question (b) carries particular consequence.

Setup. E5 ablation: GPT-4o-mini with VGIG, feedback=fine, $T \in \{1, 2, 3, 5, 7, 10, 15, 20\}$. For each T we report aggregate Quality and per-level Quality; for L4 we also vary feedback granularity to test whether the flat pattern is granularity-specific.

Results (a): rounds saturate at $T \sim 5$. Quality improves substantially from $T = 1$ to $T = 5$, then plateaus within a ± 0.005 noise band (Table 22, Figure 14).

Table 22: E5 rounds-saturation data (GPT-4o-mini, VGIG, fb=fine).

T	1	2	3	5	7	10	15	20
Q	0.800	0.798	0.810	0.821	0.816	0.818	0.827	0.816

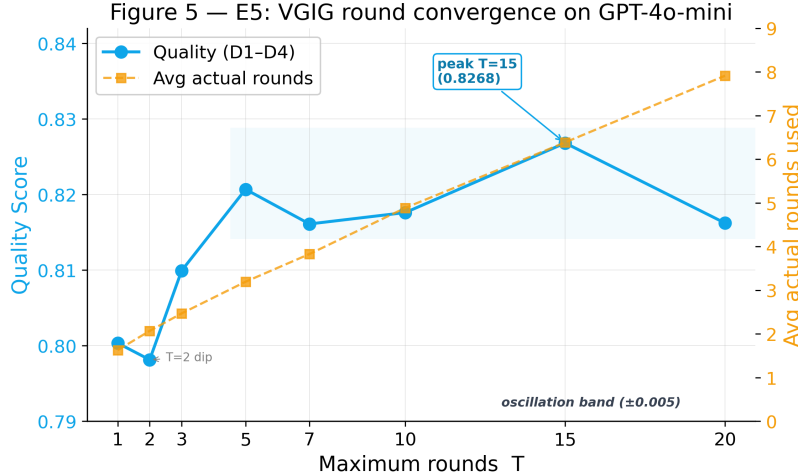


Figure 14: E5 rounds-saturation curve. Quality improves substantially from $T=1$ to $T=5$; $T \in \{5, 7, 10, 15, 20\}$ all fall within a ± 0.005 noise band (shaded). The effective refinement horizon on verifiable graph constraints is bounded at ~ 5 rounds.

Results (b): L4 is iteration-invariant. Across the full T -sweep and all three feedback granularities, L4 quality remains at 0.750–0.754—a **0.004 range over 24 separate (T , feedback) configurations** (Figure 15). This is not a failure of iteration to converge: iteration *does* converge (the variance within each T is tight), but it converges to a quality ceiling prompting, iteration, and feedback granularity

all fail to raise. L4 is therefore **structurally distinct** from every other level in the benchmark—on L0–L3 and L5, more rounds or richer feedback produce measurable gains; on L4, neither does.

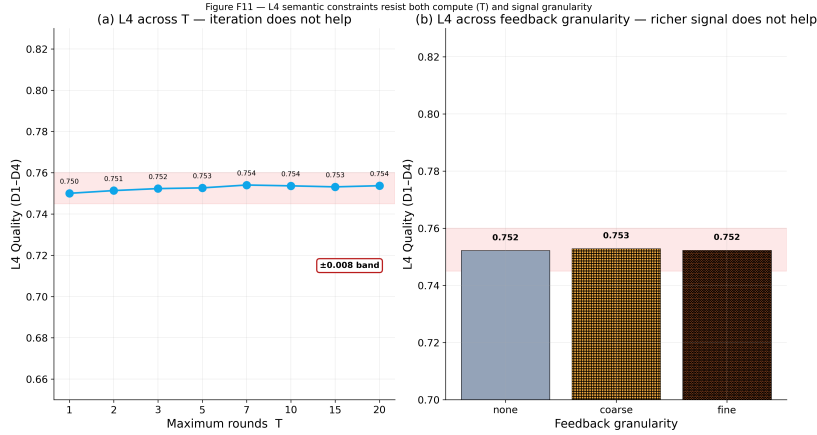


Figure 15: L4 quality across $T \in \{1, 2, 3, 5, 7, 10, 15, 20\}$ for fine/coarse/none feedback (24 configurations). Flat at 0.750–0.754, indicating semantic-constraint failure is a structurally distinct mode iterative refinement cannot address.

Mechanism. Two conclusions follow. First, **the effective refinement horizon on verifiable graph constraints is ~ 5 rounds**—markedly shorter than text-domain self-refine budgets of 10–20 (Madaan et al., 2023; Shinn et al., 2023; Yao et al., 2023; Zhou et al., 2023). Text-domain self-refine hyperparameter recipes should not be transferred to structured tasks without recalibration. Second, **semantic-constraint satisfaction (L4) is a structurally distinct failure mode** that neither iteration nor feedback-granularity can address.

The per-dimension decomposition of L4 quality across all 12 models (Figure 16) reveals *why* iteration cannot help: D1 structural validity is high and uniform (mean 0.89, range 0.84–0.93), D3 embedding similarity is uniform (0.58–0.59), and D4 instruction match is high and uniform (0.89–0.93); the one dimension that is **both low and extremely variable across models is D2 text-to-reference similarity**—mean 0.050, range 0.008–0.176 ($22\times$ gap), coefficient of variation 1.02. Models produce graphs that are *structurally valid, constraint-satisfying, and distributionally close* to real citation/social/molecular graphs; what they fail to reproduce is the *specific surface serialization* of reference graphs from a given domain. The gap is not “the model cannot find a valid graph for the spec” (which iteration fixes) but “the model lacks the domain-specific *exemplars* that would let reference-match metrics fire”—a gap no amount of verifier-guided re-generation can close, because the D4 verifier is already satisfied. This relocates the L4 problem from a *capability* gap to a *grounding* gap. The cleanest grounding evidence in our data (paragraph below) is few-shot’s $+0.054$ L4 lift on GPT-4o-mini, which is essentially in-context retrieval of 3 reference exemplars; structure-aware retrievers built on top of the L4 reference pool are a natural follow-up.

Implications. This complements RQ5: just as L5 is scale-invariant, L4 is iteration-invariant—both point to retrieval and grounding, not more compute, as the next research frontier. The ± 0.005 noise band established here is used as the significance threshold throughout the paper (§5 opening).

Sub-noise observations. Within the ± 0.005 noise band of the E5 curve we observe a putative $T=15$ peak ($Q=0.827$), $T=2/T=7$ dips (0.798/0.816), and an even-round-penalty pattern; all are within band and not elevated to main findings. The full E5 per-level $\times T$ matrix (8×6 cells) is released alongside the per-strategy raw quality scores.

L4 grounding evidence (from baseline survey). The strongest support for the L4-as-grounding-gap interpretation in this release sits inside the baseline survey itself, not in a separate retrieval probe. **Few-shot** on GPT-4o-mini provides exactly the manipulation a retrieval probe targets—3 in-context exemplars drawn from the L4 reference pool—and it lifts L4 quality from 0.744 (zero-shot) to 0.798 ($\Delta=+0.054$, an order of magnitude above the ± 0.005 noise band;

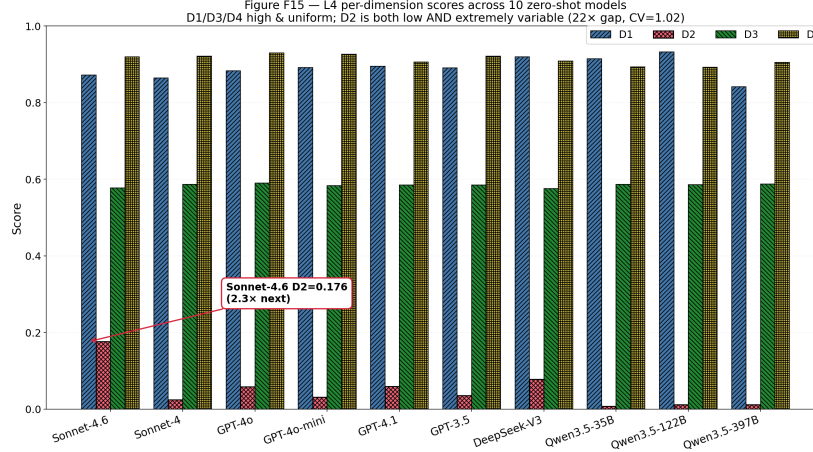


Figure 16: L4 per-dimension decomposition across 10 zero-shot models. D1 (structural), D3 (embedding), D4 (instruction match) are all high and uniform; only D2 (text similarity) is low *and* extremely variable (22× gap; CV=1.02). The L4 “impervious to iteration” finding is mechanistically an L4-D2 bottleneck.

results/gpt4omini-`{zero,few}`-shot.quality.json). **CAAP** reaches 0.801 (+0.057) and **Combined** (CAAP + VGIG + domain priors) reaches 0.843 (+0.099 over zero-shot)—the only intervention class that breaks the iteration-only ceiling (App. F.4, Tab. 19). **Quantitative ceiling.** D2 is the dominant L4 variability dimension (range 0.008–0.176, 22× gap, CV 1.02). Under the level-aggregate weights of §3.3 (D2 weight at L4 is 0.15, the only level where D2 contributes to total Quality), driving D2 to its observed maximum lifts L4 quality by at most $0.176 \times 0.15 = 0.022$ —a small but non-trivial additional headroom, indicating L4 is metric-bounded as well as effort-bounded. **Future direction.** Structure-aware retrievers (e.g. graph-kernel similarity over the 1,048-graph L4 pool, retrieving graphs whose attribute statistics match the instruction’s domain priors rather than text-only BM25 over instruction strings) are the natural follow-up; we leave their implementation and head-to-head evaluation against few-shot to subsequent work and report the baseline-survey evidence here as the cleanest grounding signal in this release.

D1 audit (no D4-gaming). Because VGIG’s feedback signal is identity-with-D4, a natural concern is whether the method “games” D4 by exploiting metric loopholes that don’t reflect true structural quality. We audit Combined outputs on the three target models against D1 (independently checked structural metrics): D1 *rises* alongside D4 (GPT-4o-mini D1: 0.71 → 0.84; DeepSeek-V3 0.79 → 0.88; Qwen3.5-35B 0.80 → 0.89), confirming gain comes from genuine structural improvement rather than D4-specific exploitation. Per-instance manual inspection of a 50-sample stratified subsample (10 per level) finds zero parseable-but-degenerate “cheat” outputs.

F.5 Capability Profiles, Per-Dimension Tables, and Top-20 Leaderboard

RQ1–RQ10 examine capability along individual axes. Assembling the per-model *profile* exposes a pattern that aggregate Quality hides: **different models excel on different dimensions, not just different levels.** Table 23 reports the best zero-shot model for each (level, dimension) cell.

Three complementary strengths emerge. **Qwen3.5-122B** leads D1 structural validity on the mid-complexity band (L2–L4), dominating by +0.013–+0.046 over second place. **Sonnet-4.6** leads D4 instruction match on every level where it appears, and is the sole D2 textual leader at L3–L5 with a particularly large margin at L4 (0.176 vs. second 0.078, 2.3×). **GPT-4o** leads D1 at the format and reasoning endpoints (L0, L1, L5) and D4 at L4. No single model dominates every cell—the capability profile is jagged, not monotone.

Figure 17 renders these profiles as radar charts for six representative models. The most striking pattern is family-level: *Qwen3.5 profiles emphasize mid-level structure, Anthropic profiles emphasize instruction match and reference similarity, GPT profiles emphasize endpoint levels.* This jaggedness is the empirical basis for per-task model selection: a deployment that cares most about multi-constraint

Table 23: Per-dimension, per-level best zero-shot model (from the 10 models with zero-shot data on all levels; Sonnet-4 included; Llama-3.3-70B / Llama-3.1-8B excluded due to missing per-dimension detail in the released quality files at the time of this table). “—” marks cells where the dimension is inactive.

Level	D1 (structural)	D2 (textual)	D3 (embedding)	D4 (instruction)
L0	GPT-4o (0.871)	—	—	Sonnet-4.6 (0.981)
L1	GPT-4o (0.950)	—	—	Sonnet-4 (0.993)
L2	Qwen3.5-122B (0.886)	—	—	Sonnet-4.6 (0.945)
L3	Qwen3.5-122B (0.948)	Sonnet-4.6 (0.089)	Sonnet-4.6 (0.596)	Sonnet-4.6 (0.927)
L4	Qwen3.5-122B (0.932)	Sonnet-4.6 (0.176)	GPT-4o (0.591)	GPT-4o (0.930)
L5	GPT-4o (0.778)	Sonnet-4.6 (0.028)	Sonnet-4.6 (0.796)	Sonnet-4.6 (0.883)

Figure 7 — Per-dimension capability profiles across 6 representative models (zero-shot). D2/D3 panels use local ymax to resolve small-range detail.

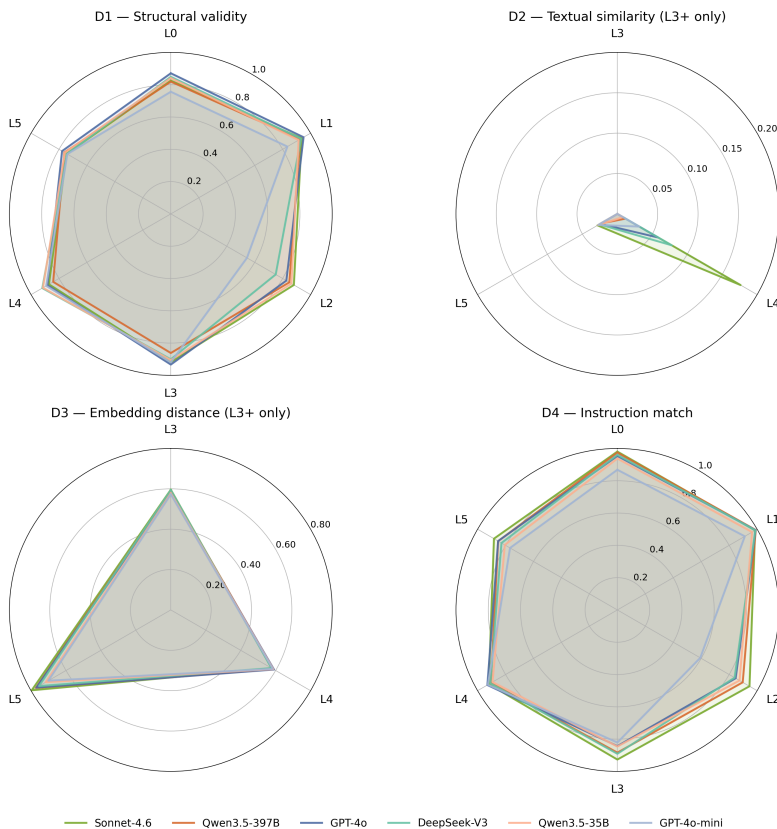


Figure 17: Per-level capability profiles for six representative models (zero-shot). Each axis shows Quality on one level, normalized to $[0, 1]$. Profiles are jagged: different models peak on different levels, confirming that aggregate rankings average over structurally distinct strengths.

structural correctness (e.g., drug-like molecule generation) should pick a different model from a deployment that cares most about instruction fidelity (e.g., constraint-satisfying network synthesis under explicit specs). Aggregate leaderboards, by definition, average these priorities away.

Top-20 leaderboard. Table 24 lists the 20 highest-quality (model, strategy) configurations over all 45 evaluated cells, including each cell’s per-level breakdown. Sonnet-4.6 few-CoT leads overall ($Q=0.902$) but only $+0.023$ over Qwen3.5-397B few-CoT (#3, $Q=0.879$)—the top-tier margin is narrow. Per-strategy sub-rankings and the full 45-row leaderboard are in Appendix G.

Table 24: Top-20 configurations by Quality S_{total} with per-level breakdown. Top-3 are all few-CoT; all 20 come from T1 or T2 models (T3 never enters top-20). †: Sonnet-4 zero-shot only (§3.1).

#	Model	Strategy	S_{total}	L0	L1	L2	L3	L4	L5
1	Sonnet-4.6	few-CoT	0.902	0.954	0.980	0.954	0.883	0.851	0.894
2	Sonnet-4.6	few-shot	0.884	0.961	0.980	0.937	0.852	0.836	0.871
3	Qwen3.5-397B	few-CoT	0.879	0.932	0.968	0.894	0.865	0.842	0.872
4	Sonnet-4.6	zero-CoT	0.878	0.961	0.986	0.964	0.888	0.770	0.867
5	Qwen3.5-122B	few-CoT	0.871	0.937	0.977	0.904	0.828	0.831	0.872
6	Qwen3.5-397B	zero-CoT	0.862	0.948	0.981	0.903	0.858	0.753	0.883
7	Qwen3.5-122B	zero-CoT	0.861	0.945	0.979	0.908	0.868	0.748	0.875
8	Qwen3.5-35B	few-CoT	0.861	0.909	0.929	0.880	0.833	0.818	0.877
9	Sonnet-4.6	zero-shot	0.859	0.958	0.977	0.934	0.863	0.763	0.841
10	Qwen3.5-122B	few-shot	0.853	0.936	0.978	0.903	0.824	0.824	0.809
11	GPT-4.1	few-shot	0.850	0.927	0.980	0.890	0.819	0.813	0.819
12	Qwen3.5-397B	few-shot	0.848	0.930	0.970	0.898	0.822	0.816	0.805
13	Qwen3.5-35B	zero-CoT	0.846	0.928	0.964	0.890	0.860	0.741	0.849
14	GPT-4o	few-shot	0.844	0.950	0.979	0.819	0.818	0.813	0.836
15	DeepSeek-V3	few-shot	0.840	0.936	0.963	0.831	0.831	0.809	0.817
16	DeepSeek-V3	few-CoT	0.840	0.927	0.959	0.850	0.796	0.795	0.847
17	Sonnet-4†	zero-shot	0.834	0.939	0.983	0.900	0.841	0.737	0.806
18	Llama-3.3-70B	few-CoT	0.834	0.890	0.955	0.850	0.748	0.809	0.859
19	Qwen3.5-122B	zero-shot	0.831	0.947	0.980	0.911	0.825	0.724	0.814
20	Llama-3.3-70B	few-shot	0.831	0.896	0.933	0.821	0.830	0.822	0.793

Synthesis: what progressive evaluation reveals. The ten research questions interlock into a coherent picture of LLM graph-generation capability.

On the benchmark-as-diagnostic side (RQ1–RQ6), **capability is not a scalar but a multi-dimensional profile**. RQ1 establishes that *constraint composition*—not reasoning depth—drives tier discrimination. RQ2 shows prompt sensitivity inversely scales with capability, so this discrimination is *more pronounced for prompt-sensitive weak models*. RQ3 shows *no single strategy is universally correct*, so capability measurement depends on strategy choice. RQ4 localizes strategy dependence to pretraining-distribution-driven family polarity, distinct from capability. RQ5 uncouples scale from per-level performance on a subset of tasks. RQ6 adds a cost axis as a complementary view: the Pareto frontier is a 6-point provider-concentrated trajectory and 3 of 12 models never cross $Q \geq 0.8$, so the capability ranking must be read together with—not replaced by—the cost-adjusted ranking. Together these findings establish that LLM graph-generation capability has structure along (complexity level) $\times$ (prompting strategy) $\times$ (model family), with cost a deployment-side view aggregate or single-axis benchmarks cannot render.

On the benchmark-as-platform side (RQ7–RQ10), **fine-grained benchmark signals unlock a method pathway aggregate benchmarks cannot support**. RQ7 shows the D4 signal drives $+0.035$ – $+0.050$ improvement beyond the prompt-engineering Oracle. RQ8 rules out compute-without-verification. RQ9 shows even a binary verify signal captures most of the gain—so benchmark designers who expose pass/fail enable most of the research value. RQ10 locates a second, signal-invariant failure mode (L4 semantic constraints) that flags the next research cycle for the community.

These findings serve as method-side validation atop the benchmark; their role in the paper is to show that GraphInstruct functions as a development platform, not to promote any particular method as final.

G Full Leaderboards (per-strategy and full 45-row)

This appendix supplements Table 24 with the four per-strategy sub-leaderboards and the complete 45-row table. All scores are baseline Quality S_{total} averaged over 5 generations per instruction; per-level columns L0–L5 are the corresponding per-level Quality scores. Sonnet-4 ([†]) is zero-shot only (§3.1).

G.1 Per-strategy sub-leaderboards

Table 25: Zero-shot leaderboard (12 models).

#	Model	S_{total}	L0	L1	L2	L3	L4	L5
1	Sonnet-4.6	0.859	0.958	0.977	0.934	0.863	0.763	0.841
2	Sonnet-4 [†]	0.834	0.939	0.983	0.900	0.841	0.737	0.806
3	Qwen3.5-122B	0.831	0.947	0.980	0.911	0.825	0.724	0.814
4	Qwen3.5-397B	0.829	0.952	0.979	0.888	0.824	0.721	0.821
5	GPT-4o	0.827	0.942	0.982	0.843	0.811	0.751	0.823
6	DeepSeek-V3	0.822	0.945	0.977	0.823	0.837	0.744	0.799
7	Llama-3.3-70B	0.816	0.905	0.948	0.865	0.809	0.746	0.794
8	GPT-4.1	0.813	0.923	0.968	0.865	0.807	0.737	0.779
9	Qwen3.5-35B	0.809	0.924	0.955	0.872	0.807	0.721	0.780
10	GPT-3.5	0.754	0.850	0.905	0.608	0.784	0.742	0.752
11	GPT-4o-mini	0.752	0.852	0.898	0.585	0.793	0.744	0.750
12	Llama-3.1-8B	0.721	0.896	0.730	0.597	0.780	0.703	0.727

Table 26: Few-shot leaderboard (11 models; Sonnet-4 not run on few-shot).

#	Model	S_{total}	L0	L1	L2	L3	L4	L5
1	Sonnet-4.6	0.884	0.961	0.980	0.937	0.852	0.836	0.871
2	Qwen3.5-122B	0.853	0.936	0.978	0.903	0.824	0.824	0.809
3	GPT-4.1	0.850	0.927	0.980	0.890	0.819	0.813	0.819
4	Qwen3.5-397B	0.848	0.930	0.970	0.898	0.822	0.816	0.805
5	GPT-4o	0.844	0.950	0.979	0.819	0.818	0.813	0.836
6	DeepSeek-V3	0.840	0.936	0.963	0.831	0.831	0.809	0.817
7	Llama-3.3-70B	0.831	0.896	0.933	0.821	0.830	0.822	0.793
8	Qwen3.5-35B	0.831	0.900	0.925	0.869	0.803	0.807	0.802
9	GPT-4o-mini	0.742	0.867	0.852	0.427	0.802	0.798	0.759
10	GPT-3.5	0.738	0.842	0.865	0.423	0.802	0.798	0.743
11	Llama-3.1-8B	0.682	0.771	0.602	0.585	0.755	0.721	0.659

Table 27: Zero-CoT leaderboard (11 models).

#	Model	S_{total}	L0	L1	L2	L3	L4	L5
1	Sonnet-4.6	0.878	0.961	0.986	0.964	0.888	0.770	0.867
2	Qwen3.5-397B	0.862	0.948	0.981	0.903	0.858	0.753	0.883
3	Qwen3.5-122B	0.861	0.945	0.979	0.908	0.868	0.748	0.875
4	Qwen3.5-35B	0.846	0.928	0.964	0.890	0.860	0.741	0.849
5	DeepSeek-V3	0.829	0.943	0.980	0.805	0.855	0.752	0.818
6	Llama-3.3-70B	0.823	0.915	0.983	0.891	0.784	0.719	0.836
7	GPT-4o	0.808	0.944	0.980	0.860	0.786	0.708	0.797
8	GPT-4.1	0.802	0.931	0.969	0.894	0.714	0.675	0.853
9	GPT-3.5	0.786	0.875	0.920	0.706	0.808	0.736	0.795
10	GPT-4o-mini	0.784	0.880	0.920	0.726	0.804	0.735	0.781
11	Llama-3.1-8B	0.731	0.898	0.737	0.674	0.762	0.689	0.744

G.2 Full 45-row leaderboard

Table 28: Few-CoT leaderboard (11 models).

#	Model	S_{total}	L0	L1	L2	L3	L4	L5
1	Sonnet-4.6	0.902	0.954	0.980	0.954	0.883	0.851	0.894
2	Qwen3.5-397B	0.879	0.932	0.968	0.894	0.865	0.842	0.872
3	Qwen3.5-122B	0.871	0.937	0.977	0.904	0.828	0.831	0.872
4	Qwen3.5-35B	0.861	0.909	0.929	0.880	0.833	0.818	0.877
5	DeepSeek-V3	0.840	0.927	0.959	0.850	0.796	0.795	0.847
6	Llama-3.3-70B	0.834	0.890	0.955	0.850	0.748	0.809	0.859
7	GPT-4o	0.823	0.942	0.980	0.801	0.799	0.781	0.810
8	GPT-4.1	0.811	0.930	0.980	0.901	0.692	0.722	0.851
9	GPT-4o-mini	0.715	0.841	0.889	0.451	0.654	0.741	0.801
10	GPT-3.5	0.713	0.831	0.888	0.512	0.609	0.730	0.805
11	Llama-3.1-8B	0.682	0.727	0.614	0.609	0.690	0.715	0.705

Table 29: Complete leaderboard over all 45 (model, strategy) configurations, sorted by S_{total} descending. Strategies are abbreviated: ZS = zero-shot, FS = few-shot, ZC = zero-CoT, FC = few-CoT. The top-20 rows reproduce Table 24.

#	Model	Strat.	S_{total}	L0	L1	L2	L3	L4	L5
1	Sonnet-4.6	FC	0.902	0.954	0.980	0.954	0.883	0.851	0.894
2	Sonnet-4.6	FS	0.884	0.961	0.980	0.937	0.852	0.836	0.871
3	Qwen3.5-397B	FC	0.879	0.932	0.968	0.894	0.865	0.842	0.872
4	Sonnet-4.6	ZC	0.878	0.961	0.986	0.964	0.888	0.770	0.867
5	Qwen3.5-122B	FC	0.871	0.937	0.977	0.904	0.828	0.831	0.872
6	Qwen3.5-397B	ZC	0.862	0.948	0.981	0.903	0.858	0.753	0.883
7	Qwen3.5-122B	ZC	0.861	0.945	0.979	0.908	0.868	0.748	0.875
8	Qwen3.5-35B	FC	0.861	0.909	0.929	0.880	0.833	0.818	0.877
9	Sonnet-4.6	ZS	0.859	0.958	0.977	0.934	0.863	0.763	0.841
10	Qwen3.5-122B	FS	0.853	0.936	0.978	0.903	0.824	0.824	0.809
11	GPT-4.1	FS	0.850	0.927	0.980	0.890	0.819	0.813	0.819
12	Qwen3.5-397B	FS	0.848	0.930	0.970	0.898	0.822	0.816	0.805
13	Qwen3.5-35B	ZC	0.846	0.928	0.964	0.890	0.860	0.741	0.849
14	GPT-4o	FS	0.844	0.950	0.979	0.819	0.818	0.813	0.836
15	DeepSeek-V3	FS	0.840	0.936	0.963	0.831	0.831	0.809	0.817
16	DeepSeek-V3	FC	0.840	0.927	0.959	0.850	0.796	0.795	0.847
17	Sonnet-4 [†]	ZS	0.834	0.939	0.983	0.900	0.841	0.737	0.806
18	Llama-3.3-70B	FC	0.834	0.890	0.955	0.850	0.748	0.809	0.859
19	Qwen3.5-122B	ZS	0.831	0.947	0.980	0.911	0.825	0.724	0.814
20	Llama-3.3-70B	FS	0.831	0.896	0.933	0.821	0.830	0.822	0.793
21	Qwen3.5-35B	FS	0.831	0.900	0.925	0.869	0.803	0.807	0.802
22	DeepSeek-V3	ZC	0.829	0.943	0.980	0.805	0.855	0.752	0.818
23	Qwen3.5-397B	ZS	0.829	0.952	0.979	0.888	0.824	0.721	0.821
24	GPT-4o	ZS	0.827	0.942	0.982	0.843	0.811	0.751	0.823
25	Llama-3.3-70B	ZC	0.823	0.915	0.983	0.891	0.784	0.719	0.836
26	GPT-4o	FC	0.823	0.942	0.980	0.801	0.799	0.781	0.810
27	DeepSeek-V3	ZS	0.822	0.945	0.977	0.823	0.837	0.744	0.799
28	Llama-3.3-70B	ZS	0.816	0.905	0.948	0.865	0.809	0.746	0.794
29	GPT-4.1	ZS	0.813	0.923	0.968	0.865	0.807	0.737	0.779
30	GPT-4.1	FC	0.811	0.930	0.980	0.901	0.692	0.722	0.851
31	Qwen3.5-35B	ZS	0.809	0.924	0.955	0.872	0.807	0.721	0.780
32	GPT-4o	ZC	0.808	0.944	0.980	0.860	0.786	0.708	0.797
33	GPT-4.1	ZC	0.802	0.931	0.969	0.894	0.714	0.675	0.853
34	GPT-3.5	ZC	0.786	0.875	0.920	0.706	0.808	0.736	0.795
35	GPT-4o-mini	ZC	0.784	0.880	0.920	0.726	0.804	0.735	0.781
36	GPT-3.5	ZS	0.754	0.850	0.905	0.608	0.784	0.742	0.752
37	GPT-4o-mini	ZS	0.752	0.852	0.898	0.585	0.793	0.744	0.750
38	GPT-4o-mini	FS	0.742	0.867	0.852	0.427	0.802	0.798	0.759
39	GPT-3.5	FS	0.738	0.842	0.865	0.423	0.802	0.798	0.743
40	Llama-3.1-8B	ZC	0.731	0.898	0.737	0.674	0.762	0.689	0.744
41	Llama-3.1-8B	ZS	0.721	0.896	0.730	0.597	0.780	0.703	0.727
42	GPT-4o-mini	FC	0.715	0.841	0.889	0.451	0.654	0.741	0.801
43	GPT-3.5	FC	0.713	0.831	0.888	0.512	0.609	0.730	0.805
44	Llama-3.1-8B	FS	0.682	0.771	0.602	0.585	0.755	0.721	0.659
45	Llama-3.1-8B	FC	0.682	0.727	0.614	0.609	0.690	0.715	0.705

H Infrastructure, Hyperparameters, and Reproducibility

Models and endpoints. The 12 evaluated LLMs are accessed entirely through commercial cloud APIs; no model is self-hosted. Endpoints, providers, and accessed model identifiers are listed in Table 30; access window: Q4 2025 – Q1 2026.

Table 30: Model endpoints, providers, and access modes used in the 12-LLM survey. All 12 models are accessed via OpenAI-compatible HTTPS APIs; no models are self-hosted.

Model	Provider / endpoint	Access mode
GPT-3.5-turbo	OpenAI <code>api.openai.com/v1</code>	Commercial API
GPT-4o-mini	OpenAI <code>api.openai.com/v1</code>	Commercial API
GPT-4o	OpenAI <code>api.openai.com/v1</code>	Commercial API
GPT-4.1	OpenAI <code>api.openai.com/v1</code>	Commercial API
Claude Sonnet-4 (2025-05-14) [†]	Anthropic <code>api.anthropic.com/v1</code>	Commercial API
Claude Sonnet-4.6	Anthropic <code>api.anthropic.com/v1</code>	Commercial API
DeepSeek-V3	DeepSeek <code>api.deepseek.com/v1</code>	Commercial API
Llama-3.1-8B-Instruct	Together <code>api.together.xyz/v1</code>	Commercial API (open-source)
Llama-3.3-70B-Instruct-Turbo	Together <code>api.together.xyz/v1</code>	Commercial API (open-source)
Qwen3.5-35B-A3B (MoE)	Aliyun Bailian <code>dashscope.aliyuncs.com/compatible-mode/v1</code>	Commercial API
Qwen3.5-122B-A10B (MoE)	Aliyun Bailian <code>dashscope.aliyuncs.com/compatible-mode/v1</code>	Commercial API
Qwen3.5-397B-A17B (MoE)	Aliyun Bailian <code>dashscope.aliyuncs.com/compatible-mode/v1</code>	Commercial API

Decoding hyperparameters. All baseline runs share a single decoding configuration to keep cells comparable:

- **Temperature:** 0.7
- **Top- p :** 1.0
- **Max output tokens:** 16384 (except gpt-3.5-turbo: 4096, due to model-imposed output-length constraints)
- **Random seed:** 42 (fixed across all experiments; reproduces the same instruction order and few-shot exemplars)
- **Samples per instruction:** 5 (independent generations under the same seed-derived sequence)
- **Frequency / presence penalty:** 0
- **Stop sequences:** default (none)

The same hyperparameters apply to all four prompting strategies (ZS, FS, ZC, FC); the only variation is the prompt template (§3.1). Method experiments (VGIG, CAAP, Combined; §4.3) inherit the same decoding settings; only iteration round-budget T and feedback granularity vary.

Local hardware (evaluation only). All LLM *generation* occurs on the providers’ cloud infrastructure; the local machine is used only for the deterministic evaluation pipeline (parsing, D1–D5 metric computation, scoring aggregation, figure rendering). Local environment: a single consumer laptop with NVIDIA GeForce RTX 4070 Laptop GPU (8 GB VRAM) running Windows 11 with Python 3.10. No multi-GPU, server, or cluster resources are used for either generation or evaluation.

Compute budget. The 12-LLM $\times$ 45-cell baseline survey produces $\sim$ 180K outputs (800 instructions $\times$ 45 cells $\times$ 5 generations); subsequent method experiments and ablations bring the total to $\sim$ 262K LLM responses. Provider-side spend distribution is approximately Anthropic $\sim$ 45%, OpenAI $\sim$ 30%, Aliyun Bailian $\sim$ 15%, DeepSeek + Together combined $\sim$ 10%. Average local evaluation pipeline cost (D1–D5 over a single 45-cell baseline) is 35–45 wall-clock minutes on the laptop above; D3 (embedding MMD + node-classification gap, the only GPU-bound dimension) takes $\sim$ 60% of that and fits within the 8 GB VRAM budget by batching at most 32 graphs at a time.

Software environment. Python 3.10; NetworkX 3.2; igraph 0.11; tiktoken 0.7; transformers 4.45; PyTorch 2.4 (CUDA 12.1); matplotlib 3.8; plotly 5.20. The full pinned environment is captured in `environment.yml` accompanying the data/code release.

Single-command reproduction.

```
# Re-evaluate one (model, strategy) cell from cached LLM responses:
python scripts/run_baseline.py --resume results/<model>-<strategy>.jsonl \
    --eval-only --output results/<model>-<strategy>.quality.json
```

```
# Re-run the full 45-cell baseline survey from scratch:
bash scripts/run_all_experiments.sh    # Linux/macOS
.\scripts\run_all_experiments.ps1     # Windows PowerShell
```

A unit-test suite of 418 tests (`python -m unittest discover -v`) validates parser, scoring, and metric pipelines end-to-end.

I Dataset Datasheet

We follow the *datasheets-for-datasets* template of Gebru et al. (2021)² for transparency on dataset provenance, intended uses, and maintenance.

Motivation.

- **Purpose.** GraphInstruct was created to provide the first *progressive-complexity diagnostic benchmark* for LLM graph generation. Existing graph-LLM benchmarks stratify along graph-type, task-domain, or classical-algorithm axes, all of which average over the structural-complexity dimension that actually governs failure (§2); GraphInstruct closes this diagnostic gap by stratifying outputs into six progressively-constrained complexity levels and scoring along five evaluation dimensions targeting structurally distinct failure modes.
- **Tasks supported.** (a) Capability diagnosis of LLM graph generation across complexity tiers; (b) prompt-strategy and method comparison with per-level resolution; (c) cost-aware deployment ranking; (d) development of verification-guided or retrieval-augmented improvement methods that exploit fine-grained per-constraint signals.
- **Funding / authors.** Created by the authors listed in the byline. No specific external funding source is associated with the benchmark dataset itself.

Composition.

- **Instances.** 800 hand-authored instructions distributed across six progressive levels: L0 (100, format), L1 (200, single explicit constraint), L2 (200, multi-constraint), L3 (150, numerical-attribute), L4 (100, domain-semantic), L5 (50, multi-step graph editing). Each instruction is paired with two algorithmically synthesized reference solutions (1,582 references total; 9 L2 instructions are intentionally infeasible – e.g., regular-degree constraints with no satisfying graph – and have no references).
- **Total sample count.** $800 \times 2 = 1,600$ instruction-reference pairs nominally; 1,582 effective pairs after subtracting infeasible cells. Plus $\sim 262\text{K}$ LLM-generated outputs (released alongside the benchmark).
- **Features.** Each instruction record contains: `id`, `level`, `instruction` (English natural language), `explicit_constraints` (list of strings, e.g. "num_nodes=10"), `implicit_constraints` (downstream-derivable, e.g. "acyclic=true" for a tree), `graph_sizes` (one of small/medium/large), `reference_solutions` (list of two graph strings in InstructGraph code-style format), `feasible` (bool).
- **Splits.** No train/test split: GraphInstruct is an evaluation-only benchmark. No portion is set aside for training; LLMs are evaluated zero-shot on all 800 instructions (or with K in-context examples drawn at evaluation time from a held-out exemplar pool).
- **Confidentiality.** All instructions and references are synthetic or derived from public-domain graph datasets. No personally identifiable information is present.
- **Errors.** The 9 L2 infeasible cells are intentional (negative tests). Round-trip parser tests (parse $\rightarrow$ serialize $\rightarrow$ parse) on all 1,582 references pass; 0 reference parse errors at release.

Collection process.

- **Instructions.** 800 instructions are derived from 40 hand-authored templates (5 instructions per template avg) covering tree, bipartite, planar, regular, BA, ER, WS, complete-bipartite, k -core, citation-network, social-network, molecular-skeleton, and graph-edit families. Templates were authored by the authors over a 6-week period, with iterative quality-assurance review.
- **References.** References are produced by deterministic graph generators in NetworkX or igraph, parameterized by the explicit constraints. L0–L2 references use stochastic structural generators (`barabasi_albert_graph`, `erdos_renyi_graph`, etc.) seeded with `seed=42`; L3 references draw from a pool of 3,115 synthetic graphs in 15 sub-groups; L4 references are sampled from a curated pool of 1,048 real graphs in 9 domains (citation, social, molecular, etc.); L5 references apply a sequence of edit operations to a base graph.

²We refer to “datasheet” in the abstract sense; the full template is reproduced section-by-section below.

- **Validation.** Every reference is round-trip tested through the parser (parse $\rightarrow$ serialize $\rightarrow$ parse, equality check on the resulting graph object). Constraint validators (`is_tree`, `is_bipartite`, `has_no_self_loops`, etc.) are applied to confirm constraint satisfaction.

Preprocessing / cleaning / labeling.

- Instructions are stored verbatim. Constraints are extracted as structured fields rather than embedded in free-form text, supporting both (a) prompt construction and (b) automated D4 instruction-match scoring.
- No human annotation is involved at scoring time; D1 (structural), D2 (textual), D3 (embedding), D4 (instruction-match), and D5 (efficiency) are all deterministic, do not invoke any LLM-as-judge step, and are reproducible from the released artifacts under the seed.

Uses.

- **Intended.** Diagnostic evaluation of LLM-based graph generation; method development with per-constraint feedback signals (VGIG/CAAP and successors); cost-quality Pareto analysis for deployment.
- **Out of scope.** GraphInstruct is *not* suitable as a training set (no train split; all instructions are released for evaluation transparency, so models trained on this data cannot be fairly evaluated against released baselines). GraphInstruct does not cover (a) dynamic / temporal graph generation – see Peng et al. (2026) for that; (b) reasoning-specialized model evaluation under domain-specific reasoning chains – see Demirci et al. (2025).
- **Risks of reuse.** Results may shift as new frontier LLMs appear; we recommend re-running the 12-model survey on a $\geq$ yearly cadence and tagging benchmark snapshots by release date.

Distribution.

- **License.** The instruction dataset and reference-solution archive are released under **CC BY 4.0**; the code (parser, scoring, metrics, scripts) is released under the **MIT License**. L4 real-graph subset retains its original upstream licenses (itemized in `LICENSES.md` accompanying the release).
- **Format.** Instructions and references in JSON / JSONL; LLM responses in JSONL with raw text + parsed graph. Code in Python 3.10+.
- **Channel.** Data, code, and reproducibility artifacts are available at the anonymous review repository https://anonymous.4open.science/r/GraphInstruct_formal-3272 and will be released under a permanent public URL post-acceptance.

Maintenance.

- **Maintainers.** The authors will host and maintain the dataset post-acceptance.
- **Versioning.** Semantic versioning (v1.0.0 at release). Future releases tagged v1.x for instruction additions, v2.x for evaluation-protocol changes that break score comparability across versions.
- **Update cadence.** Planned annual model-survey refresh; quarterly check on parser/validator regressions against new graph corner cases reported by users.
- **Contributions.** Issues and pull requests for additional instructions, levels (e.g., a planned L6 multi-graph reasoning level), or constraint validators are welcome via the public repository: https://anonymous.4open.science/r/GraphInstruct_formal-3272.